MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

A SPARSITY-AWARE NEAR-MEMORY SpMVM ACCELERATOR FOR EDGE INFERENCE ON FPGA

Minor Specialization Report

Submitted by

Vemana Venkata Pujithram
Registration No.: 230907016

(Individual Minor Specialization Project)

Under the guidance of

Dr. Rahul Ratnakumar
Department of Electronics and Communication Engineering, MIT Manipal

in partial fulfilment of the requirements for the award of the degree of

MINOR SPECIALIZATION IN

Embedded Systems

MANIPAL INSTITUTE OF TECHNOLOGY
(A Constituent Unit of Manipal Academy of Higher Education)
MANIPAL-576104, KARNATAKA, INDIA

October 2026

ACKNOWLEDGMENTS

I express my sincere gratitude to my project guide, Dr. Rahul Ratnakumar, Department of Electronics and Communication Engineering, Manipal Institute of Technology, Manipal, for his constant guidance, critical feedback and encouragement throughout this work. His insistence on connecting every architectural decision to a measurable hardware quantity shaped the way this project was formulated, implemented and reported, and the discussions that led to the conference paper on which this report builds were invaluable.

I thank the Head of the Department, the project coordinators and the faculty of the Department of Electronics and Communication Engineering, MIT Manipal, for the academic environment, the laboratory facilities and the software licences that made the SystemVerilog design, Vivado simulation and synthesis experiments possible. I also thank the laboratory staff for their technical assistance with the development environment.

I am grateful to my classmates and friends for the many discussions on sparse linear algebra, FPGA memory resources and verification practice, and to my family for their patience and support during the semester. Finally, I acknowledge the authors of the open literature and the vendor documentation cited in this report, on whose work the background and the comparisons presented here depend.

Vemana Venkata Pujithram

ABSTRACT

Deploying deep neural networks on edge devices is limited by the cost of memory access and by the wasted arithmetic spent on the zero weights that survive pruning. This project presents the design, verification and post-synthesis characterisation of a Sparsity-Aware Near-Memory Sparse Matrix–Vector Multiplication (SpMVM) accelerator for edge inference on an AMD/Xilinx Zynq-7000 SoC (xc7z020clg484-1). True analog in-memory computing relies on access to SRAM bit-lines that commercial FPGA fabrics do not expose; the architecture therefore realises the practical FPGA equivalent, BRAM-centric near-memory computing, by placing a zero-skipping controller and a signed INT8 multiply–accumulate (MAC) unit directly at the synchronous read ports of three on-chip block RAMs that hold the compressed weights, the column indices and the dense input vector.

Weights are stored in Compressed Sparse Row (CSR) form so that only non-zero elements are fetched and multiplied. A five-state finite state machine (IDLE, FETCH_PTR, FETCH_VEC, COMPUTE, DONE_STATE) sequences each stored non-zero in exactly three clock cycles, absorbing the one-cycle BRAM read latency of the column-index bank that feeds the indirect address of the vector bank; the resulting latency is $T = 1 + 3 \cdot NNZ + 1$ cycles. The arithmetic unit multiplies two signed 8-bit operands into an exact 16-bit product and accumulates into a 32-bit register without DSP48E1 slices. The design is verified with a directed MAC testbench and a top-level testbench that preloads the BRAMs and reproduces the expected accumulation $0 \rightarrow 15 \rightarrow 7$ for the dot product $5(3) + (-4)(2)$.

Out-of-context synthesis on the Zynq-7000 device reports 154 slice LUTs, 41 slice registers (FDCE), 3 RAMB18E1 block RAMs and 0 DSP48E1 slices, a 10.32 ns critical path delay (about 96.9 MHz), and a vectorless post-synthesis power estimate of 4.502 W total on-chip power (4.235 W dynamic, 0.268 W static). At 90 % sparsity the core delivers an effective dense-equivalent throughput of about 0.64 GOPS. The report also derives storage and latency models for Dense, CSR, COO and CSC formats and lays out a system-level roadmap covering multi-row CSR sequencing, AXI4-Lite wrapping, Zynq PS–PL integration and board deployment.

Keywords: FPGA, Sparse Matrix–Vector Multiplication, CSR, Near-Memory Computing, Compute-in-Memory, Block RAM, INT8 MAC, Zynq-7000, SystemVerilog.

TABLE OF CONTENTS

Section	Page
Acknowledgments	i
Abstract	ii
List of tables	iv
List of figures	v
List of abbreviations	vi
Chapter 1 Introduction	1
Chapter 2 Literature review and objectives	4
Chapter 3 Methodology	8
Chapter 4 Contribution of student	41
Chapter 5 Result analysis	43
Chapter 6 Conclusion and future scope	54
Visible outcome	57
References	58
Minor specialization project details	61

LIST OF TABLES

No	Table title	Page
1.1	Layers of the design and their role in this report	3
2.1	Literature lessons and corresponding project objectives	7
3.1	Target platform, toolchain and design parameters	8
3.2	Storage formats for an $M \times N$ matrix with NNZ non-zeros	11
3.3	Computation and data-movement cost, dense versus CSR (64 × 64 INT8 tile, 90 % sparsity)	16
3.4	Parameters and signal widths of the Sparse-CIM core	17
3.5	States of the five-state sparsity-aware controller	23
3.6	RTL modules and testbenches	29
3.7	Acceptance checks for the Sparse-CIM core	33
3.8	Proposed AXI4-Lite register map of the Sparse-CIM IP	36
3.9	Bitstream export and embedded-software workflow	38
5.1	Cycle-by-cycle simulation values for the directed test ($5 \times 3 + (-4) \times 2$); t is the start time of each 10 ns cycle	44
5.2	FPGA resource utilisation of the Sparse-CIM core on the Zynq-7000 (xc7z020clg484-1)	47
5.3	Timing results and derived quantities (post-synthesis, out-of-context)	48
5.4	Vectorless post-synthesis power estimate (xc7z020clg484-1, out-of-context)	49
5.5	Latency, speedup and effective throughput versus sparsity (64 × 64 INT8 tile, F = 96.9 MHz)	51
5.6	Comparison with existing sparse accelerators and MAC architectures (from the iSES paper)	52
5.7	System-scaling enhancements and verification gates	53
6.1	Visible technical deliverables	57

LIST OF FIGURES

No	Figure title	Page
3.1	Development workflow of the Sparse-CIM accelerator	9
3.2	SpMVM using CSR representation for the 4 × 4 example (from the iSES paper): *A*, *x*, the three CSR arrays and the zero-skipping computation	10
3.3	System-level block diagram of the Sparse-CIM engine (from the iSES paper): the FSM controller routes addresses to three BRAM banks and operands to the MAC unit	13
3.4	State-transition diagram of the sparsity-aware FSM (from the iSES paper)	20
3.5	Datapath micro-architecture of the MAC unit (from the iSES paper): 8-bit signed multiplier, 16-bit product, 32-bit accumulator register with feedback	26
3.6	Proposed Zynq-7000 integration: ARM Cortex-A9 PS, AXI interconnect, AXI4-Lite wrapper, AXI BRAM controllers and the Sparse-CIM PE with dual-port banks	37
5.1	Behavioural simulation of the top module (from the iSES paper): BRAM preload, FSM zero-skipping sequence and accumulation 0 → 15 → 7	43
5.2	Storage of a 64 × 64 INT8 weight tile for Dense, COO and CSR formats versus sparsity	46
5.3	Post-synthesis out-of-context power estimate (from the iSES paper): total on-chip power 4.502 W	49
5.4	Cycle-count speedup of the zero-skipping PE versus sparsity for a 64 × 64 tile (two baselines)	51

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ADC	Analog-to-Digital Converter
AXI	Advanced eXtensible Interface
BRAM	Block Random Access Memory
CE	Clock Enable
CIM	Compute-in-Memory
CNN	Convolutional Neural Network
COO	Coordinate (storage format)
CSC	Compressed Sparse Column
CSR	Compressed Sparse Row
DMA	Direct Memory Access
DRAM	Dynamic Random Access Memory
DSP	Digital Signal Processing (slice)
ECG / EEG	Electrocardiogram / Electroencephalogram
FDCE	D flip-flop with Clock Enable and asynchronous Clear
FF	Flip-Flop
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
GOPS	Giga Operations Per Second
HBM	High Bandwidth Memory
IMC	In-Memory Computing
INT8	8-bit signed integer
IP	Intellectual Property block
LUT	Look-Up Table
MAC	Multiply–Accumulate
MLP	Multi-Layer Perceptron
NMC	Near-Memory Computing
NNZ	Number of Non-Zero elements
OOC	Out-Of-Context (synthesis)
PE	Processing Element
PL / PS	Programmable Logic / Processing System
RAMB18E1	7-series 18 Kb Block RAM primitive
RTL	Register-Transfer Level
SoC	System on Chip
SpMV / SpMVM	Sparse Matrix–Vector Multiplication
SRAM	Static Random Access Memory
TB	Testbench
XSA	Xilinx Support Archive (hardware handoff file)

CHAPTER 1
INTRODUCTION

1.1 Background and problem statement

Matrix–vector multiplication is the central computation of neural-network inference: every fully connected layer, the projection layers of a Transformer, and the lowered form of a convolution all reduce to evaluating $y = Ax$, where A is an $M \times N$ weight matrix and x is an N-element activation vector [1]. The elements of the result are $y_i = \sum_j A_{ij}\, x_j$. After training, weight pruning removes connections whose magnitude is small, and sparsity levels of 50–90 % or higher are routinely reached with limited accuracy loss [2], [3]. The pruned matrix A therefore contains mostly zeros.

A conventional dense datapath ignores this structure. It stores every zero, fetches every zero from memory and spends a clock cycle and the associated dynamic energy multiplying by it [4]. The waste has two components. The first is arithmetic: for a matrix of density $\rho = \mathrm{NNZ}/(MN)$, a fraction $1 - \rho$ of all multiply–accumulate operations has no effect on the result. The second, and usually more expensive, is data movement. Fetching one weight from off-chip DRAM costs orders of magnitude more energy than the multiplication that consumes it [5], [4]; on an edge device with a battery or a few-watt thermal budget this “memory wall”, rather than the arithmetic itself, bounds the achievable inferences per joule.

The engineering problem addressed in this project is therefore stated as follows: design a compact hardware engine for $y = Ax$ with a pruned, quantised A that (i) keeps all operand traffic on-chip, (ii) performs work proportional to the number of non-zero weights rather than to MN, and (iii) maps onto the resources of a commercial low-cost FPGA with a minimal arithmetic footprint. The target platform is the AMD/Xilinx Zynq-7000 SoC (xc7z020clg484-1) [6], [7], which offers 53,200 LUTs, 106,400 flip-flops, 280 RAMB18E1 block RAMs and 220 DSP48E1 slices alongside a dual-core ARM Cortex-A9 processing system.

1.2 Motivation and relevance

Four properties of FPGA-based inference motivate the approach taken here. **Determinism:** an FPGA executes a fixed finite state machine against synchronous memories, so the latency of an inference is a closed-form function of the stored data and not of cache state, operating-system scheduling or DRAM refresh. This is the property that makes FPGAs attractive for in-vehicle, robotic and medical workloads with hard real-time deadlines. **On-chip locality:** the Zynq-7020 provides about 4.9 Mb of block RAM distributed in columns across the fabric [8]; a pruned layer tile held in these RAMs never touches DRAM during inference. **Low precision:** INT8 quantisation of weights and activations is well established for inference [9], [10], and an 8×8 signed multiplier is small enough to be built from look-up tables, avoiding the DSP48E1 slices that are sized for 25×18-bit products [11]. **Sparsity:** storing and

processing only non-zero weights converts the pruning ratio directly into reduced storage, reduced memory traffic and reduced cycle count.

A further motivation is conceptual. The research literature on *in-memory computing* (IMC) [12], [13], [14] describes computing inside the memory array itself, which removes the weight-movement cost entirely. True analog IMC is, however, a property of custom SRAM or non-volatile memory macros whose bit-lines are accessible to the designer. On a commercial FPGA the block RAM is a hard primitive exposing only a synchronous address–data interface. The closest practical realisation is *near-memory computing* (NMC): placing the compute logic immediately at the memory read port so that every operand is consumed in the cycle after it is read, with no intermediate buffering and no off-chip transfer. Section 3.5 develops this distinction rigorously, and the design in this report is a BRAM-centric NMC engine in that sense.

1.3 Main engineering contribution

The contribution of this project is a modular, parameterisable **BRAM-centric Sparse Compute-in-Memory / Near-Memory Processing Engine** (the Sparse-CIM PE, top module `sparse_cim_top`) together with its complete verification and hardware characterisation. The engine integrates the following elements:

- **Compressed weight and index banks.** Two `bram_bank` instances (`bank_values` and `bank_col_idx`) hold the non-zero INT8 weights and their column indices, addressed by a common non-zero pointer.
- **Indirect vector fetching.** A third bank, `bank_vector`, holds the dense input vector and is read at the address delivered by the column-index bank, so that the activation matching each non-zero weight is fetched without scanning the vector.
- **A five-state zero-skipping controller.** `sparse_cim_controller` sequences IDLE, FETCH_PTR, FETCH_VEC, COMPUTE and DONE_STATE; because it iterates over `nnz_count` stored non-zeros, zero weights consume neither cycles nor memory reads.
- **A DSP-free signed INT8 MAC.** `mac_unit` forms an exact 16-bit signed product and accumulates into a 32-bit register under an enable and a synchronous clear, and synthesises entirely into LUTs and flip-flops.
- **Verification and characterisation.** A directed MAC testbench and a top-level testbench demonstrate correct signed accumulation, and Vivado out-of-context synthesis quantifies area (154 LUTs, 41 FFs, 3 RAMB18E1, 0 DSPs), delay (10.32 ns) and vectorless power (4.502 W).

1.4 Scope and evolution of the design

The project is organised around a mathematical formulation, an implemented compute core and a scaling framework. The mathematical formulation (Section 3.4) defines the CSR representation of *A* by its `values`, `col_idx` and `row_ptr` arrays and proves the equivalence of the zero-skipping computation to the dense product. The implemented core evaluates the dot product of one compressed row with the dense vector and accumulates it in the 32-bit register; this is the unit that is simulated, synthesised and characterised. The scaling framework (Sections 3.10, 3.13 and 6.4) extends the same datapath with `row_ptr` sequencing, result write-back, an AXI4-Lite register interface and Zynq PS–PL integration. Table 1.1 summarises how these layers relate.

Table 1.1 Layers of the design and their role in this report

Layer	Architecture described	Role in this report
Mathematical model	$y = Ax$ with CSR arrays `values`, `col_idx`, `row_ptr`; zero-skipping dot products	Formulation, storage and cycle models (Sections 3.4, 3.6, 5.9)
Implemented core	Three BRAM banks, 5-state FSM, INT8 MAC with 32-bit accumulator; one compressed row per invocation	Simulated, synthesised and characterised (Chapters 3 and 5)
Scaling framework	6-state multi-row CSR FSM: IDLE → FETCH_ROW_PTR → STREAM_CSR → MAC_ACCUM → WRITEBACK → DONE	Architectural extension (Sections 3.10, 3.13)
SoC integration	AXI4-Lite wrapper, AXI BRAM controllers, Zynq PS (ARM Cortex-A9), .xsa export, Vitis driver	Integration roadmap (Sections 3.20–3.23, 6.4)

1.5 Demonstration boundary

The demonstrated results of this project are of two kinds. **Behavioural simulation** in the Vivado Simulator exercises the RTL with directed stimulus and shows the expected accumulation and the `done` handshake. **Post-synthesis characterisation** on the Zynq-7000 device (`xc7z020clg484-1`) reports resource utilisation, critical-path delay and vectorless power for the out-of-context core, which isolates the accelerator from I/O pads and from the rest of a system. Together these establish the correctness of the datapath and the hardware cost of the engine.

System-level deployment — the AXI4-Lite register interface, the Zynq processing-system integration, bitstream export and execution on a physical board — forms the integration roadmap of the project. It is specified in Sections 3.20–3.23 so that the next phase can proceed directly from this report, and it is the natural continuation of the work rather than part of the verified results presented in Chapter 5.

CHAPTER 2
LITERATURE REVIEW AND OBJECTIVES

2.1 Pruned neural networks and sparse inference

Han *et al.* showed that a trained network can be pruned by removing low-magnitude weights and retraining, reducing the parameter count of large networks by an order of magnitude without loss of accuracy [2]. Subsequent work established a taxonomy of sparsity patterns [3]. *Unstructured* sparsity removes individual weights anywhere in a matrix; it achieves the highest compression for a given accuracy but yields irregular access patterns that dense hardware cannot exploit. *Structured* sparsity removes whole rows, columns, filters or fixed-size blocks, or enforces a pattern such as 2:4 (two non-zeros in every group of four), which regularises the index stream at some cost in flexibility [15].

The three principal layer types differ in how sparsity appears. In **fully connected (FC)** and multi-layer perceptron layers the weight matrix is used directly as A, and unstructured pruning ratios above 90 % have been reported for large FC layers [2], so SpMVM is the exact computation. In **convolutional (CNN)** layers, filters are lowered to matrix form (im2col) or processed with a sparse dataflow such as the Cartesian-product scheme of SCNN [16]; both reduce to products of sparse weight vectors with dense activation windows. In **recurrent and Transformer** layers, the gate and projection matrices are large dense-activation products to which magnitude pruning applies in the same way, and the resulting matrix–vector products dominate single-batch edge inference, where the activation vector is dense and the weight matrix is sparse — precisely the SpMVM case treated in this report.

2.2 Efficient sparse accelerators and memory-centric computing

EIE. The Efficient Inference Engine of Han *et al.* is an ASIC accelerator that operates directly on a compressed network [17]. It stores the pruned, weight-shared matrix in an interleaved compressed-column format in on-chip SRAM distributed across processing elements, and skips zero activations dynamically. Its principal lesson for the present work is that keeping the entire compressed model in on-chip memory removes DRAM from the inference path, and that the index and pointer overhead of the compressed format must be accounted for in the energy model.

SCNN. Parashar *et al.* describe an accelerator for compressed-sparse CNNs that stores both weights and activations in compressed form and multiplies all non-zero weight–activation pairs in a Cartesian-product array, reporting performance and energy gains over a comparably provisioned dense accelerator [16]. The lesson is that sparsity must be exploited *and* the resulting irregular accumulation must be managed in hardware.

GraphLily and SERPENS. GraphLily is an overlay for graph linear algebra on HBM-equipped FPGAs that implements SpMV and related kernels using 19 HBM channels [18]. SERPENS is an HBM-based SpMV accelerator using 24 HBM channels and wide streaming processing elements to reach high throughput [19]. Both target data-centre cards: in the comparison of this report they are listed at 43 W and 48 W respectively. Their lesson is that SpMV throughput on large matrices is bandwidth-limited and that HBM parallelism is the data-centre answer — an answer unavailable within an edge power budget.

Cerberus. Hwang *et al.* propose a triple-mode accelerator that switches between compressed, bitmap and dense representations according to the sparsity of the operand, trading format overhead against skipping efficiency [20]; it is listed at 17.032 W and 112.4 GOPS in the comparison. The lesson is that no single format is optimal across sparsity levels, which motivates the break-even analysis of Section 3.4.

Zhang *et al.* and Abhiram *et al.* Zhang *et al.* present a 109-GOPS/W FPGA Vision Transformer accelerator with a weight-loop dataflow, reporting about 348 GOPS at 3.2 W using roughly 120K LUTs [21]; it is the closest INT8 FPGA full-system comparison. Abhiram *et al.* report a fault-resilient INT8 MAC unit for mission-critical AI hardware at 2967 μm^2, 0.507 mW and 3.99 ns [22]; it is an ASIC arithmetic-only comparison point for the MAC datapath. Survey literature on FPGA sparse accelerators [23], [24], on MAC design [25], [26], and on SpMV optimisation for general-purpose processors [27], [28] frames these designs.

Dataflow and FPGA streaming designs. Eyeriss introduces a row-stationary dataflow that minimises the movement of weights, activations and partial sums by keeping reusable data in local storage next to the arithmetic [29]; FINN builds streaming FPGA inference pipelines in which each layer has its own datapath and on-chip memory [30]. Neither is a sparse SpMV engine, but both support the design principle adopted here: place the memory that holds the operands directly beside the arithmetic unit that consumes them.

ASIC in-memory computing versus FPGA BRAM near-memory computing

In ASIC **analog SRAM IMC**, multiple word-lines are activated simultaneously and the bit-line current or charge sums the products of stored weights and applied inputs; an ADC at the column periphery digitises the result [12], [14]. Compute and storage are the same physical structure, so weights do not move at all. The price is a custom bit-cell and periphery (word-line drivers, ADCs, calibration), limited precision and sensitivity to process variation. In **FPGA BRAM near-memory computing**, the memory is a fixed synchronous macro whose bit-lines are inaccessible; computation happens in programmable logic that is adjacent to the read port. Weights still move, but only a few hundred picoseconds to a nanosecond of programmable routing, and only on-chip. The result is exact digital arithmetic, deterministic timing and full reconfigurability at the price of a read-out energy that analog IMC avoids.

2.3 Numerical precision for edge inference

Integer-only inference with 8-bit weights and activations preserves accuracy for most vision and language models when quantisation-aware calibration is used [9], [10]. In a signed two's-complement INT8 representation each operand lies in [−128, 127]. The product of two INT8 operands lies in [−16,256, 16,384]: the extreme values are (−128)(127) = −16,256 and (−128)(−128) = +16,384. Both fit in a 16-bit signed number (range −32,768 to 32,767), so a 16-bit product is *exact* and no rounding or truncation occurs at the multiplier output.

The accumulator must hold a sum of up to NNZ such products. Since $|\text{product}| \le 2^{14}$, a 32-bit signed accumulator ($|\text{value}| \le 2^{31} - 1$) cannot overflow for fewer than $2^{17} = 131{,}072$ terms; the addressing of the present instance (6-bit pointers, at most 63 non-zeros per invocation) leaves a margin of more than 2,000×. The trade-off against narrower accumulators is a handful of extra flip-flops and carry-chain slices, which is negligible against the guarantee of overflow-free accumulation for any practical layer tile.

2.4 FPGA platform and implementation literature

The Zynq-7000 family integrates a dual-core ARM Cortex-A9 processing system (PS) with 7-series programmable logic (PL) on one die [6]. The xc7z020 device provides 53,200 six-input LUTs, 106,400 flip-flops, 140 36 Kb block RAMs (equivalently 280 RAMB18E1) and 220 DSP48E1 slices [7]. Each **RAMB18E1** is an 18 Kb true dual-port synchronous memory whose output has a one-cycle read latency (optionally two with the output register enabled) and that can be configured from 16K × 1 up to 512 × 36 and, for byte-oriented data, as 2K × 9 [8]. Because the read is registered inside the macro, the BRAM presents data one clock edge after the address — the fact on which the three-cycle schedule of Section 3.11 rests.

A **DSP48E1** slice contains a 25 × 18-bit signed multiplier, a 48-bit accumulator and a pre-adder [11]. For 8-bit operands most of this capability is unused; an 8 × 8 signed multiplier built from LUTs and carry chains [31] costs on the order of one hundred LUTs, leaves the DSP columns free and, because LUTs are placed anywhere in the fabric, allows the multiplier to sit adjacent to a BRAM column without the placement constraints of the DSP columns. Vivado synthesis infers block RAM from the `ram_style` attribute and from the synchronous-read coding style, and infers LUT/carry arithmetic when DSP use is not requested [32]. Timing is reported against constraints [33] and power is estimated either vectorlessly or from switching activity [34]; Section 3.17–3.19 and Chapter 5 apply these flows.

2.5 Project objectives

The literature review leads to the following objectives for the project:

1. Formulate SpMVM $y = Ax$ over CSR data and derive storage, traffic and cycle models for Dense, CSR, COO and CSC formats.

2. Design a BRAM-centric near-memory Sparse-CIM PE in SystemVerilog comprising three parameterised BRAM banks, a five-state zero-skipping FSM and a signed INT8 MAC with 32-bit accumulation.
3. Derive the exact cycle-accurate execution latency of the controller, including the one-cycle synchronous BRAM read latency.
4. Verify the arithmetic and the control flow by directed simulation, including a negative weight and the `start`/`done` handshake.
5. Characterise area, critical-path delay and vectorless power of the core on the Zynq-7000 `xc7z020clg484-1` using Vivado out-of-context synthesis, with no DSP48E1 usage.
6. Compare the proposed engine with published sparse accelerators and MAC units on a common table of scope, precision, area, power, delay and throughput.
7. Specify a system-level integration roadmap: multi-row CSR sequencing, AXI4-Lite IP packaging, Zynq PS–PL integration, Vitis control software and edge deployment scenarios.

Table 2.1 Literature lessons and corresponding project objectives

Source	**Lesson drawn**	**Project objective**
Han *et al.* (pruning) [2]	Pruned layers are mostly zeros; sparsity is exploitable	Objective 1 (CSR formulation)
EIE [17]	Keep the compressed model in on-chip memory; count index overhead	Objectives 1, 2 (BRAM banks, storage model)
SCNN [16]	Manage irregular accumulation of sparse products in hardware	Objectives 2, 4 (FSM and MAC)
GraphLily, SERPENS [18], [19]	HBM bandwidth is a data-centre answer; edge needs on-chip locality	Objectives 2, 5, 6 (on-chip NMC, power)
Cerberus [20]	Format choice depends on sparsity level	Objective 1 (break-even analysis)
Zhang *et al.* [21]	INT8 FPGA systems reach high throughput at large area	Objective 6 (comparison)
Abhiram *et al.* [22]	INT8 MAC reference at ASIC level	Objectives 5, 6
Quantisation [9], [10]	INT8 preserves accuracy; wide accumulation needed	Objectives 2, 4 (16-bit product, 32-bit sum)
Xilinx guides UG473, UG901, UG907 [8], [32], [34]	RAMB18E1 timing, synthesis inference, power flow	Objectives 3, 5
Zynq TRM, AXI [6], [35]	PS–PL integration via AXI4-Lite and BRAM controllers	Objective 7 (roadmap)

CHAPTER 3
METHODOLOGY

3.1 Type and nature of the project

This is an individual hardware-design and hardware-characterisation project in the domain of digital VLSI and computer architecture. Its method is analytical and experimental: the computation is first formulated mathematically (CSR storage and the zero-skipping dot product), then realised as synthesisable SystemVerilog RTL, then verified by directed behavioural simulation, and finally characterised by post-synthesis area, timing and power analysis on a specific commercial device, the AMD/Xilinx Zynq-7000 `xc7z020clg484-1`.

The project is a *design-space* study as much as a design: the choice of storage format, of memory organisation (near-memory rather than in-memory), of control schedule (three cycles per non-zero) and of arithmetic mapping (LUT rather than DSP) are each justified quantitatively in this chapter and evaluated in Chapter 5. The scope of the verified results is the Sparse-CIM processing element; the system-level integration needed for a complete SoC product is specified as a roadmap in Sections 3.20–3.23.

3.2 Hardware and software requirements

Table 3.1 lists the platform, tool and parameter choices used in the project, together with the resources that the integration roadmap will additionally require.

Table 3.1 Target platform, toolchain and design parameters

Item	Specification	Purpose
Target device	AMD/Xilinx Zynq-7000 SoC `xc7z020clg484-1` (speed grade –1)	Synthesis and characterisation target
Device resources	53,200 LUTs; 106,400 FFs; 280 RAMB18E1 (140 × 36 Kb); 220 DSP48E1	Denominators for utilisation percentages
Design tool	AMD/Xilinx Vivado Design Suite (synthesis, out-of-context mode)	Synthesis, timing and power estimation
Simulator	Vivado Simulator (xsim)	Behavioural verification of RTL
Language	SystemVerilog (IEEE 1800-2017) [36] with `` `timescale 1ns / 1ps ``	RTL and testbenches
Design parameters	`DATA_WIDTH` = 8; `ADDR_WIDTH` = 6; `DEPTH` = 64	Operand width and bank size
Memory inference	`(* ram_style = "block" *)` on each bank array	Force RAMB18E1 inference
Simulation clock	100 MHz (`always #5 clk = ~clk`, period 10 ns)	Behavioural simulation only
Roadmap hardware	Zynq-7000 board with ARM Cortex-A9 PS; JTAG/SD boot	Board deployment
Roadmap software	Vivado IP Integrator, Vitis (bare-metal) or PetaLinux	Software control of the IP

3.3 Development workflow

Figure 3.1 shows the development workflow. The solid boxes are the steps carried out and reported in this project; the dashed boxes are the system-integration steps that continue from the same RTL baseline. The flow is deliberately linear for the implemented part: each stage produces an artefact (a mathematical model, RTL files, a waveform, a utilisation/timing/power report) that is the input to the next, which keeps every reported number traceable to a specific source file and tool run.

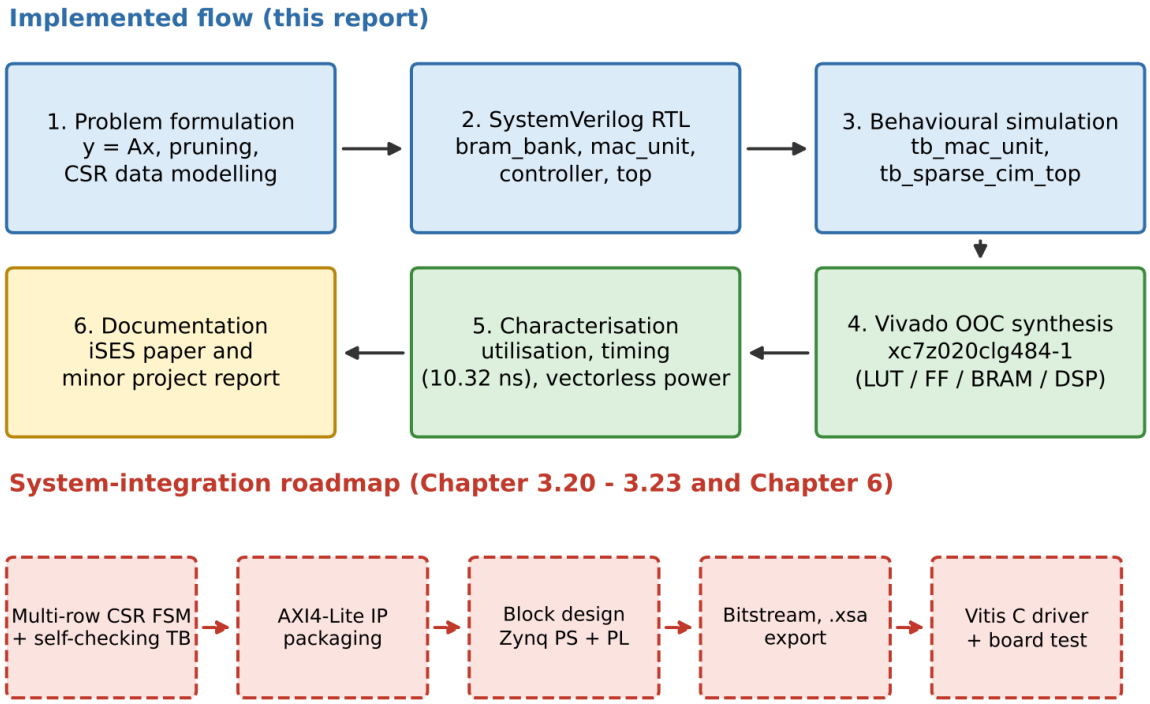


Figure 3.1 Development workflow of the Sparse-CIM accelerator

3.4 SpMVM formulation and sparse-data preparation

Let $A \in \mathbb{Z}^{M\times N}$ be the INT8 weight matrix, $x \in \mathbb{Z}^{N}$ the INT8 activation vector and NNZ the number of non-zero entries of A. The dense product is

$$y_i = \sum_{j=0}^{N-1} A_{ij}\, x_j, \quad i = 0, \ldots, M-1 \tag{3.1}$$

CSR representation. Compressed Sparse Row storage [37] represents A by three one-dimensional arrays: `values` (length NNZ), the non-zero elements in row-major order; `col_idx` (length NNZ), the column index of each stored value; and `row_ptr` (length $M + 1$), where `row_ptr[i]` is the index in `values` of the first non-zero of row i and `row_ptr[M]` = NNZ. The number of non-zeros in row i is `row_ptr[i+1]` − `row_ptr[i]`. With these arrays the product becomes

$$y_i = \sum_{k=\texttt{row_ptr}[i]}^{\texttt{row_ptr}[i+1]-1} \texttt{values}[k] \cdot x_{\texttt{col_idx}[k]} \tag{3.2}$$

Equivalence. Every term of the dense sum with $A_{ij} = 0$ contributes zero and may be omitted without changing y_i; every term with $A_{ij} \neq 0$ is stored exactly once, at some position k in row i's segment, with `col_idx[k]` = j. Hence the two sums are identical, and the number of multiplications falls from MN to NNZ.

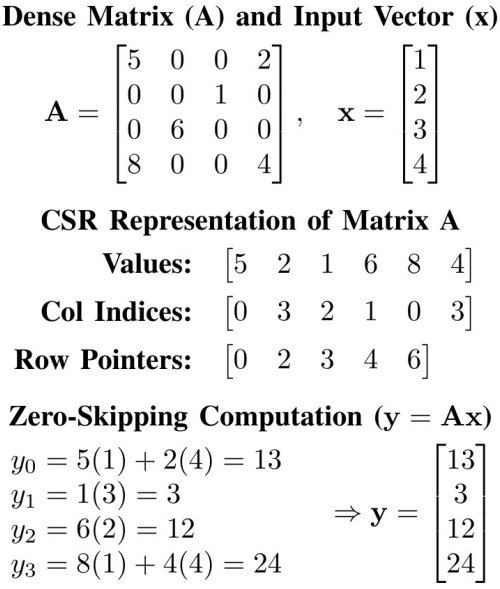


Figure 3.2 SpMVM using CSR representation for the 4 × 4 example (from the iSES paper): A, x, the three CSR arrays and the zero-skipping computation

Worked example (Figure 3.2). For the 4 × 4 matrix A with rows (5, 0, 0, 2), (0, 0, 1, 0), (0, 6, 0, 0), (8, 0, 0, 4) and $x = (1, 2, 3, 4)^T$, NNZ = 6, `values` = [5, 2, 1, 6, 8, 4], `col_idx` = [0, 3, 2, 1, 0, 3] and `row_ptr` = [0, 2, 3, 4, 6]. Evaluating row by row gives $y_0 = 5(1) + 2(4) = 13$, $y_1 = 1(3) = 3$, $y_2 = 6(2) = 12$ and $y_3 = 8(1) + 4(4) = 24$, so $y = (13, 3, 12, 24)^T$. Six multiplications replace sixteen; the ten skipped operations are exactly the multiplications by zero.

Storage bit-width derivation

Let b_v be the bit-width of a stored value (8 for INT8), and define the minimum index widths

$$b_c = \lceil \log_2 N \rceil, \quad b_r = \lceil \log_2 M \rceil, \quad b_p = \lceil \log_2 (NNZ + 1) \rceil \tag{3.3}$$

where b_c addresses a column, b_r a row and b_p a position in the `values` array (a row pointer must be able to take any value in 0…NNZ). The storage of the four formats is then

$$S_{Dense} = MN\, b_v \tag{3.4}$$

$$S_{CSR} = NNZ\, b_v + NNZ\, b_c + (M + 1)\, b_p \tag{3.5}$$

$$S_{COO} = NNZ\, (b_v + b_r + b_c) \tag{3.6}$$

$$S_{CSC} = NNZ\, b_v + NNZ\, b_r + (N + 1)\, b_p \tag{3.7}$$

For the 4 × 4 example ($b_c = b_r = 2$, $b_p = 3$) these evaluate to $S_{\text{Dense}} = 128$ bits, $S_{\text{CSR}} = 48 + 12 + 15 = 75$ bits, $S_{\text{COO}} = 6(8 + 2 + 2) = 72$ bits and $S_{\text{CSC}} = 48 + 12 + 15 = 75$ bits. The

implemented banks use byte-wide index words (the BRAM word is 8 bits regardless of the minimum index width), so the physical CSR footprint of this tiny example is 6·8 + 6·8 + 5·8 = 136 bits; the saving appears for the larger, sparser tiles for which the core is intended (Section 3.6 and Chapter 5).

Index-overhead threshold

CSR is smaller than dense storage when $S_{\mathrm{CSR}} < S_{\mathrm{Dense}}$. Writing the density ρ = NNZ/(MN) and neglecting the row-pointer term (which is (M + 1) b_p, small compared with NNZ values for tiles that are not hypersparse), the condition is NNZ ($b_v + b_c$) < $MN\, b_v$, that is

$$\rho < \rho^* \approx b_v / (b_v + b_c) \tag{3.8}$$

For b_v = 8 and a 64-column tile (b_c = 6), ρ* = 8/14 ≈ 57 %, i.e. CSR wins at sparsity above about 43 %. With byte-wide index words (b_c = 8) ρ* = 50 %. The COO break-even is lower, ρ^*_{COO} = 8/(8 + 6 + 6) = 40 %, because COO stores a row index for every non-zero instead of a short row-pointer array. CSR is smaller than COO whenever (M + 1) b_p < NNZ b_r, which holds for every tile with more than a few non-zeros per row; COO is preferable only for hypersparse matrices with NNZ ≈ M or smaller. CSC mirrors CSR with the roles of rows and columns exchanged.

Table 3.2 Storage formats for an $M \times N$ matrix with NNZ non-zeros

Format	Arrays stored	Storage (bits)	Row-wise SpMVM fit
Dense	all MN elements	$MN\, b_v$	Trivial, but multiplies every zero
CSR	`values`, `col_idx`, `row_ptr` (M + 1)	NNZ($b_v + b_c$) + (M + 1)b_p	Best: one row yields one y_i with a local accumulator
COO	`values`, `row_idx`, `col_idx`	NNZ($b_v + b_r + b_c$)	Needs row-change detection; larger index stream
CSC	`values`, `row_idx`, `col_ptr` (N + 1)	NNZ($b_v + b_r$) + (N + 1)b_p	Column-major: scatter-accumulates into y (random writes)

CSR is chosen for this design for a structural reason in addition to the storage comparison of Table 3.2. The row-wise dataflow produces each y_i in one contiguous run of non-zeros, so a single accumulator register suffices and the output is written once per row; CSC would require a read-modify-write of y for every non-zero. The `bank_vector` read indexed by `col_idx` is then an ordinary BRAM read.

Sparse-data preparation. Preparation is a software step performed offline (or by the host processor): quantise the pruned weights to INT8, scan A row-major, append each non-zero to `values` and its column to `col_idx`, and record the running count at each row start in `row_ptr`. The three arrays and the dense vector are then written into the BRAM banks

through Port B. For the implemented single-row core, a row's arrays occupy addresses 0… NNZ−1 and `nnz_count` = NNZ of that row.

3.5 Proposed near-memory accelerator architecture

In-memory computing versus near-memory computing on FPGAs

What true IMC is. In an analog SRAM in-memory-computing macro [12], [14], the 6T bit-cell array that stores the weights is also the multiplier array. Input activations are applied as word-line voltages or pulse widths on many rows *simultaneously*; each bit-cell conditionally discharges (or sources current onto) its bit-line according to the stored bit; the bit-line voltage or accumulated charge is therefore the analog sum $\sum w_i x_i$ of the products along a column, and an ADC at the column periphery converts it to a digital word. The same mechanism underlies resistive-memory crossbars [13]. The weights never leave the array, which is the source of the energy advantage.

Why it is native to custom ASIC macros. Doing this requires (i) a custom bit-cell and array layout in which multiple word-lines can be asserted at once without destroying the stored data, (ii) analog periphery — word-line DACs or pulse generators, column ADCs, reference generators and calibration — sized for the precision targeted, and (iii) an analysis of noise, mismatch and non-linearity over process, voltage and temperature. These are decisions made at the transistor and layout level of an ASIC.

Why it is impractical on commercial FPGAs. The block RAM of a 7-series FPGA is a hard macro whose bit-cells, bit-lines and sense amplifiers are enclosed behind a fixed synchronous interface: an address and enable go in, a registered word comes out one clock later [8]. Only one word line per port is activated per access, the bit-lines are not accessible, there is no analog periphery to repurpose, and the programmable logic around it is purely digital and clocked. Multi-row analog accumulation inside the RAM array is therefore simply not expressible in the fabric. (Digital variants of IMC, which add logic to the array, are likewise ASIC-level modifications.)

The practical FPGA equivalent: BRAM-centric near-memory computing. What an FPGA does offer is the ability to place arbitrary logic *immediately adjacent to the memory port* and to connect it with dedicated, short routing. The Sparse-CIM architecture exploits this in three ways:

- **Compute at the read port.** The controller drives the BRAM addresses and consumes the registered BRAM outputs in the very next cycle; the zero-skipping control logic (`sparse_cim_controller`) and the signed INT8 arithmetic unit (`mac_unit`) sit directly on the `douta` buses of `bank_values`, `bank_col_idx` and `bank_vector`. No FIFO, cache or off-chip transfer intervenes.

- **Dedicated ports.** Port A of every bank is reserved for the accelerator and Port B for preloading, so that once `start` is asserted the engine has uncontested access to its operands.
- **Compute proportional to data, not to the dense shape.** Because the controller walks the compressed arrays, every BRAM read and every MAC corresponds to a stored non-zero.

The result is a design that is *near* memory rather than *in* memory: the weight still traverses a few hundred picoseconds to a nanosecond of programmable routing between the RAM macro and the multiplier, and costs the read energy of the RAM. In exchange the arithmetic is exact digital INT8, timing is deterministic, the design is reconfigurable and it is realisable today on a low-cost commercial device with standard tools. This is the sense in which the design is a "Sparse Compute-in-Memory" engine on FPGA: it realises the CIM dataflow (static weights resident next to the compute, no weight traffic from DRAM) with the near-memory mechanism that the platform allows.

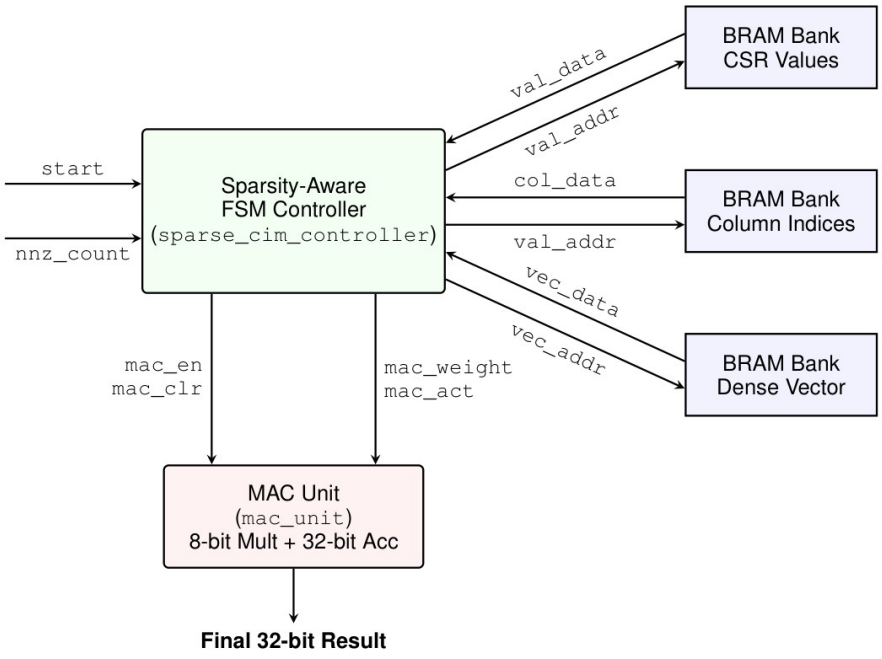


Figure 3.3 System-level block diagram of the Sparse-CIM engine (from the iSES paper): the FSM controller routes addresses to three BRAM banks and operands to the MAC unit

Figure 3.3 shows the architecture. The top module `sparse_cim_top` instantiates five blocks: one `sparse_cim_controller`, one `mac_unit` and three `bram_bank` instances. It is listed in full in Listing 3.1 and explained signal by signal below.

Listing 3.1 sparse_cim_top.sv – top-level module integrating the controller, the MAC unit and the three BRAM banks

```
1  `timescale 1ns / 1ps
2
3  module sparse_cim_top #(
4      parameter int DATA_WIDTH = 8,
5      parameter int ADDR_WIDTH = 6,
```

```
    parameter int DEPTH      = 64
)(
    input  logic clk,
    input  logic rst_n,
    input  logic start,
    input  logic [ADDR_WIDTH-1:0] nnz_count,
    output logic signed [31:0] final_result,
    output logic done,

    // ========================================
    // Backdoor Write Ports for Preloading BRAMs
    // (In a real SoC, the ARM processor would drive these)
    // ========================================
    input  logic web_val, web_col, web_vec,
    input  logic [ADDR_WIDTH-1:0] addrb,
    input  logic [DATA_WIDTH-1:0] dinb_val, dinb_col, dinb_vec
);

    // Internal wires to connect the Controller, MAC, and BRAMs
    logic [ADDR_WIDTH-1:0] val_addr, vec_addr;
    logic [DATA_WIDTH-1:0] val_data, col_data, vec_data;
    logic mac_en, mac_clr;
    logic signed [7:0] mac_weight, mac_act;

    // 1. Instantiate the Controller
    sparse_cim_controller #(
        .ADDR_WIDTH(ADDR_WIDTH),
        .DATA_WIDTH(DATA_WIDTH)
    ) controller_inst (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .nnz_count(nnz_count),
        .val_addr(val_addr),
        .vec_addr(vec_addr),
        .val_data(val_data),
        .col_data(col_data),
        .vec_data(vec_data),
        .mac_en(mac_en),
        .mac_clr(mac_clr),
        .mac_weight(mac_weight),
        .mac_act(mac_act),
        .done(done)
    );

    // 2. Instantiate the MAC Unit
    mac_unit mac_inst (
        .clk(clk),
        .rst_n(rst_n),
        .clr_acc(mac_clr),
        .en(mac_en),
        .weight_in(mac_weight),
        .act_in(mac_act),
        .acc_out(final_result)
    );

    // 3. Instantiate BRAM for Sparse Values (Weights)
    bram_bank #(.DATA_WIDTH(DATA_WIDTH), .ADDR_WIDTH(ADDR_WIDTH), .DEPTH(DEPTH)) bank_values (
        .clk(clk),
        .ena(1'b1), .addra(val_addr), .douta(val_data),
        .enb(1'b1), .web(web_val),    .addrb(addrb), .dinb(dinb_val)
    );

    // 4. Instantiate BRAM for Column Indices
    bram_bank #(.DATA_WIDTH(DATA_WIDTH), .ADDR_WIDTH(ADDR_WIDTH), .DEPTH(DEPTH)) bank_col_idx
(
        .clk(clk),
        .ena(1'b1), .addra(val_addr), .douta(col_data), // Uses same read address as values
        .enb(1'b1), .web(web_col),    .addrb(addrb), .dinb(dinb_col)
    );

    // 5. Instantiate BRAM for Dense Input Vector
    bram_bank #(.DATA_WIDTH(DATA_WIDTH), .ADDR_WIDTH(ADDR_WIDTH), .DEPTH(DEPTH)) bank_vector (
        .clk(clk),
        .ena(1'b1), .addra(vec_addr), .douta(vec_data),
        .enb(1'b1), .web(web_vec),    .addrb(addrb), .dinb(dinb_vec)
    );

endmodule
```

Port-by-port explanation of sparse_cim_top

- `clk` – common clock for the controller, MAC and all three BRAM banks (single synchronous domain).
- `rst_n` – active-low asynchronous reset, forwarded to the controller and to the MAC; the BRAM banks have no reset (their contents are preloaded).
- `start` – level-sensitive request from the host or testbench; the FSM leaves IDLE only when `start` = 1 and `nnz_count` > 0, and leaves DONE_STATE only after `start` falls.
- `nnz_count [ADDR_WIDTH-1:0]` – number of stored non-zeros to process in this invocation (at most 63 for the 6-bit instance); it must be stable while the FSM runs.
- `final_result [31:0] (signed)` – output of the 32-bit accumulator, valid while `done` = 1.
- `done` – high in DONE_STATE; tells the host that `final_result` is final.
- `web_val, web_col, web_vec` – independent Port B write enables of the value, column-index and vector banks; they let a different address be written in each bank in the same cycle phase.
- `addrb [ADDR_WIDTH-1:0]` – shared Port B write address for the three banks.
- `dinb_val, dinb_col, dinb_vec [DATA_WIDTH-1:0]` – Port B write data for the value, column-index and vector banks respectively.

Internal wires

- `val_addr` – Port A read address of `bank_values` and `bank_col_idx`; driven by the controller (equal to its non-zero pointer).
- `vec_addr` – Port A read address of `bank_vector`; driven by the controller from the column index (indirect addressing).
- `val_data, col_data, vec_data` – registered Port A outputs (`douta`) of the three banks.
- `mac_en, mac_clr` – enable and synchronous clear of the accumulator, generated by the controller.
- `mac_weight, mac_act [7:0] (signed)` – operands passed from the controller to the MAC; both are declared signed so the multiplication is two's-complement.

Instance wiring. The controller is instantiated with the same `ADDR_WIDTH` and `DATA_WIDTH` as the banks. The MAC output `acc_out` is connected directly to the top-level port `final_result`. `bank_values` and `bank_col_idx` share `val_addr` on Port A so that the weight and its column index are read together in one cycle; `bank_vector` takes the independent `vec_addr`. All three banks tie `ena` and `enb` to 1 and differ only in their write

enable and data. The comment in the source records the intended use: in a real SoC the ARM processor, through an AXI BRAM controller, would drive the Port B signals.

3.6 Sparse computational cost and data movement

Let the matrix be $M \times N$ with density ρ and sparsity $s = 1 - \rho$, so NNZ = ρMN. The number of multiply–accumulate operations is

$$C_{dense} = MN, \quad C_{CSR} = NNZ = (1 - s)\, MN \tag{3.9}$$

The ratio is exactly $1/(1 - s)$: at 90 % sparsity the engine performs one tenth of the operations. Memory traffic follows the same law. A dense engine with the vector held in BRAM reads MN weights and MN vector elements; the CSR engine reads, per non-zero, one value, one column index and one vector element, plus $M + 1$ row pointers per pass:

$$R_{dense} = 2MN \text{ words}, \quad R_{CSR} = 3\, NNZ + (M + 1) \text{ words} \tag{3.10}$$

In bits, with $b_x = 8$ for the vector and the minimum widths of Section 3.4,

$$B_{CSR} = NNZ\,(b_v + b_c + b_x) + (M + 1)\, b_p, \quad B_{dense} = MN\,(b_v + b_x) \tag{3.11}$$

Table 3.3 evaluates these expressions for a 64 × 64 INT8 tile at 90 % sparsity (NNZ = 410, $b_c = 6$, $b_p = 9$) and also gives the cycle counts of the three-cycle PE of this report (Section 5.9). The numerical instance uses `ADDR_WIDTH` larger than the 6 bits of the synthesised instance, since the 6-bit-address instance holds at most 63 non-zeros; the parameterisation is described in Section 3.8.

Table 3.3 Computation and data-movement cost, dense versus CSR (64 × 64 INT8 tile, 90 % sparsity)

Quantity	Dense	CSR (this design)	Ratio
MAC operations	4,096	410	10.0× fewer
Weight words read	4,096	410	10.0× fewer
Index words read (col_idx + row_ptr)	0	410 + 65	–
Vector words read	4,096	410	10.0× fewer
Bits read per product	65,536	9,605	6.8× fewer
Weight storage (bits)	32,768	6,325	5.2× smaller
Cycles, 3-cycle PE (T)	12,290 (dense scan)	1,232	10.0× faster

The table shows the central trade of the architecture: CSR adds an index word per non-zero and a short pointer array, but at 90 % sparsity this overhead is dwarfed by the order-of-magnitude reduction in weight words, vector words and MACs. Because all of these reads are served by on-chip BRAM, the energy of each avoided access is the BRAM read energy rather than a DRAM access, and the saving is realised without any off-chip bandwidth at all.

3.7 Test-data preparation and expected-result construction

Two data sets are used. The first is the 4 × 4 **paper example** of Figure 3.2, which validates the formulation and gives the expected output vector $y = (13, 3, 12, 24)^T$. For the single-row

core, each row is one invocation: row 0 is loaded as `values` = [5, 2], `col_idx` = [0, 3] and the vector bank as [1, 2, 3, 4] with `nnz_count` = 2, and the expected `final_result` is 5(1) + 2(4) = 13. Rows 1–3 are loaded in the same way with `nnz_count` = 1, 1 and 2, giving 3, 12 and 24. The `row_ptr` array [0, 2, 3, 4, 6] supplies the `nnz_count` of each row as the difference of successive entries and is the input of the multi-row extension (Section 3.10).

The second data set is the **directed test** used by `tb_sparse_cim_top`, chosen to exercise the two properties that matter most for the datapath: a negative weight and indirect addressing with a non-trivial column index. Two non-zero elements are stored at pointer addresses 0 and 1: weight 5 with column 0, and weight −4 with column 2. The dense vector bank holds $x[0] = 3$ and $x[2] = 2$. The expected result is

$$5 \cdot x[0] + (-4) \cdot x[2] = 5(3) + (-4)(2) = 15 - 8 = 7 \tag{3.12}$$

with the accumulator passing through the intermediate value 15 after the first non-zero. Because the second column index is 2 rather than 1, a design that simply read the vector sequentially would fetch the wrong element and produce a different result; the test therefore verifies the indirect `vec_addr` path. The expected values are constructed by hand (and are reproduced by the cycle-accurate reference model used in Section 5.1), which is appropriate for a directed test; Section 3.16 describes the automatic golden-model comparison that extends this to randomised matrices.

3.8 Parameterization and signed numerical representation

The design is parameterised by three integers, `DATA_WIDTH` = 8, `ADDR_WIDTH` = 6 and `DEPTH` = 64, with the constraint `DEPTH` ≤ $2^{\texttt{ADDR_WIDTH}}$. Changing them rescales the banks and the pointer logic without rewriting the control flow. The arithmetic widths, by contrast, are fixed by the INT8 design point: 8-bit signed operands, a 16-bit product and a 32-bit accumulator. Table 3.4 collects these quantities and their consequences.

Table 3.4 Parameters and signal widths of the Sparse-CIM core

Quantity	Value	Meaning / consequence
`DATA_WIDTH`	8	Width of every BRAM word: INT8 weight, column index, vector element
`ADDR_WIDTH`	6	Pointer, `val_addr`, `vec_addr`, `addrb`, `nnz_count` width
`DEPTH`	64	Words per bank: 512 bits, one RAMB18E1 each
Maximum NNZ per call	63	`nnz_count` is 6 bits wide, so the largest value is 63
Maximum vector length	64	`vec_addr` = `col_data[5:0]` addresses 64 entries
`mac_weight`, `mac_act`	signed [7:0]	Range −128…127, two's complement
`product`	signed [15:0]	Exact: $\lvert\text{product}\rvert \le 16{,}384 < 2^{15}$
`acc_out` / `final_result`	signed [31:0]	No overflow for fewer than 131,072 products
Controller state / pointer	3 bits / 6 bits	Together with 32 accumulator bits: 41 FDCE flip-flops

Two's-complement handling. The ports `weight_in` and `act_in` of `mac_unit` and the controller outputs `mac_weight` and `mac_act` are declared `logic signed [7:0]`, and `product` is `logic signed [15:0]`. In `product = weight_in * act_in` both operands are signed, so the expression is evaluated as a signed multiplication whose width is determined by the 16-bit target and the result is correct for negative operands. For the directed test the weight −4 is stored in the BRAM as the byte `8'hFC` (two's complement of 4); the controller assigns `mac_weight = val_data`, reinterpreting the bits as the signed value −4, and the product with 2 is the 16-bit pattern `16'hFFF8` = −8.

Sign-preserving widening. The accumulate statement `acc_out <= acc_out + 32'(product)` uses the SystemVerilog size cast. A size cast preserves the signedness of its operand, so `32'(product)` sign-extends: `16'hFFF8` becomes `32'hFFFFFFF8` = −8, and the 32-bit addition 15 + (−8) = 7 is exact. Had the product been zero-extended, the sum would be 15 + 65,528 = 65,543; the explicit cast, noted in the source as “for safety”, removes this ambiguity.

3.9 BRAM organization and operand storage

Listing 3.2 gives the complete `bram_bank` module, the parameterised dual-port memory used for all three operand arrays.

Listing 3.2 bram_bank.sv – parameterised dual-port block-RAM bank (Port A read, Port B write)

```
`timescale 1ns / 1ps

module bram_bank #(
    // Parameters make the module reusable. We can change the width/depth
    // without rewriting the core logic.
    parameter int DATA_WIDTH = 8,
    parameter int ADDR_WIDTH = 6,
    parameter int DEPTH      = 64
)(
    input  logic                  clk,

    // ==========================================
    // PORT A: Read-Only (Used by the Accelerator)
    // ==========================================
    input  logic                  ena,     // Enable Port A
    input  logic [ADDR_WIDTH-1:0] addra,   // Address to read from
    output logic [DATA_WIDTH-1:0] douta,   // Data output to the controller

    // ==========================================
    // PORT B: Write-Only (Used for Preloading Data)
    // ==========================================
    input  logic                  enb,     // Enable Port B
    input  logic                  web,     // Write Enable (1 = Write, 0 = Read)
    input  logic [ADDR_WIDTH-1:0] addrb,   // Address to write to
    input  logic [DATA_WIDTH-1:0] dinb     // Data to write into memory
);

    // This attribute tells Vivado synthesis to specifically use
    // hardware Block RAM slices on the FPGA, rather than LUTs (Registers).
    (* ram_style = "block" *) logic [DATA_WIDTH-1:0] mem [0:DEPTH-1];

    // ----------------------------------------
    // Port A Logic (Synchronous Read)
    // ----------------------------------------
    always_ff @(posedge clk) begin
        if (ena) begin
            // Data is fetched on the rising clock edge.
            // It takes 1 cycle for douta to update.
            douta <= mem[addra];
        end
```

```
    end

    // ----------------------------------------
    // Port B Logic (Synchronous Write)
    // ----------------------------------------
    always_ff @(posedge clk) begin
        if (enb) begin
            if (web) begin
                mem[addrb] <= dinb;
            end
        end
    end

endmodule
```

Signal-by-signal explanation

- `clk` – single clock for both ports.
- `ena, addra, douta (Port A)` – read port: when `ena` = 1, `mem[addra]` is loaded into the output register `douta` at each rising edge, so `douta` reflects the address presented in the *previous* cycle — a one-cycle synchronous read latency.
- `enb, web, addrb, dinb (Port B)` – write port: when both `enb` and `web` are 1, `dinb` is written to `mem[addrb]` at the rising edge; `web` = 0 performs no access.

Memory inference. The array is declared `(* ram_style = "block" *) logic [DATA_WIDTH-1:0] mem [0:DEPTH-1]`. The attribute instructs Vivado synthesis to implement the array in a block RAM rather than in distributed (LUT) RAM or flip-flops [32]. Together with the registered-read coding style of the Port A `always_ff` block (the read value passes through a register, not combinationally to the output), this maps onto the synchronous read of a RAMB18E1 with its output latch, and the write `always_ff` block maps onto the second port [8]. Each bank holds 64 × 8 = 512 bits and therefore occupies one RAMB18E1; the three banks account exactly for the three RAMB18E1 primitives reported in Chapter 5.

Port discipline. Port A is read-only and is dedicated to the accelerator (the controller never writes it); Port B is write-only and is dedicated to the preload path. Two consequences follow. First, once `start` is asserted the accelerator is the only reader and no arbitration logic is needed, which gives the deterministic single-cycle read access the schedule depends on. Second, because the memory has two independent ports on one clock, a host may in principle write the next layer's data while the current one is processed, provided the written region is not being read; the present protocol conservatively preloads before `start`.

Three instantiated banks. `sparse_cim_top` instantiates the module three times:

- `bank_values` – the CSR non-zero INT8 weights, read at `val_addr`;
- `bank_col_idx` – the column index of each non-zero, read at the same `val_addr` so weight and index arrive together;
- `bank_vector` – the dense input vector, read at `vec_addr` derived from `col_data`.

The banks have no reset and no initial contents in the RTL; they are filled through Port B before each run. A read of a never-written location returns an undefined value in simulation, and the protocol guarantees that only written locations are read (addresses below `nnz_count` and the column indices stored by the host).

3.10 Sparsity-aware control-unit FSM

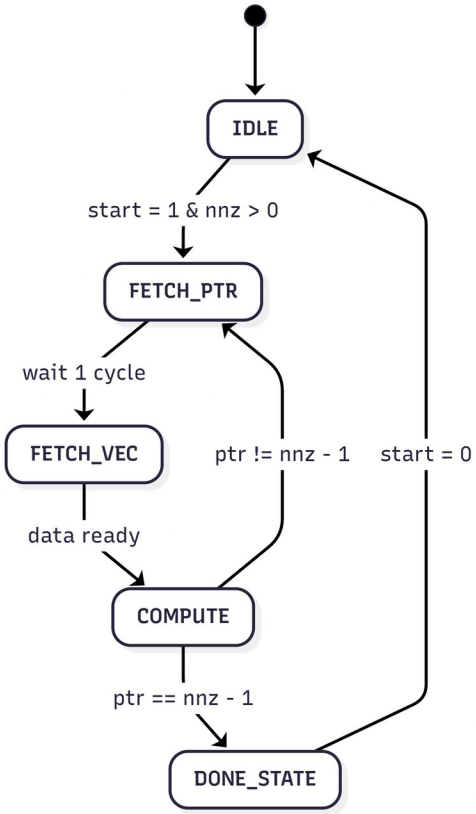


Figure 3.4 State-transition diagram of the sparsity-aware FSM (from the iSES paper)

The controller is a Moore-style five-state FSM whose structure is shown in Figure 3.4. A reset places it in IDLE. When `start` is high and `nnz_count` is non-zero it enters FETCH_PTR, then unconditionally passes through FETCH_VEC and COMPUTE; at the end of COMPUTE it either returns to FETCH_PTR for the next non-zero or, when the pointer equals `nnz_count` − 1, enters DONE_STATE. DONE_STATE holds until the host lowers `start`, then returns to IDLE. The complete source is given in Listing 3.3.

Listing 3.3 sparse_cim_controller.sv – five-state sparsity-aware controller

```
`timescale 1ns / 1ps

module sparse_cim_controller #(
    parameter int ADDR_WIDTH = 6,
    parameter int DATA_WIDTH = 8
)(
    input  logic clk,
    input  logic rst_n,
    input  logic start,

    // Tells the FSM how many elements to process before stopping
    input  logic [ADDR_WIDTH-1:0] nnz_count,

    // ========================================
```

```
    // BRAM Interface (Driving addresses, reading data)
    // ========================================
    output logic [ADDR_WIDTH-1:0] val_addr,
    output logic [ADDR_WIDTH-1:0] vec_addr,
    input  logic [DATA_WIDTH-1:0] val_data,
    input  logic [DATA_WIDTH-1:0] col_data,
    input  logic [DATA_WIDTH-1:0] vec_data,

    // ========================================
    // MAC Interface (Controlling the arithmetic)
    // ========================================
    output logic mac_en,
    output logic mac_clr,
    output logic signed [7:0] mac_weight,
    output logic signed [7:0] mac_act,

    // Status flag for the testbench
    output logic done
);

    // Define the states for our FSM
    typedef enum logic [2:0] {
        IDLE,
        FETCH_PTR,
        FETCH_VEC,
        COMPUTE,
        DONE_STATE
    } state_t;

    state_t current_state, next_state;

    // Pointer to track which non-zero element we are currently processing
    logic [ADDR_WIDTH-1:0] current_ptr;

    // ----------------------------------------
    // Sequential Block: State Memory & Counter
    // ----------------------------------------
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            current_state <= IDLE;
            current_ptr   <= 0;
        end else begin
            current_state <= next_state;

            // Only increment the pointer after a successful MAC compute
            if (current_state == COMPUTE) begin
                current_ptr <= current_ptr + 1;
            end else if (current_state == IDLE) begin
                current_ptr <= 0;
            end
        end
    end

    // ----------------------------------------
    // Combinational Block: Next State & Outputs
    // ----------------------------------------
    always_comb begin
        // Default assignments prevent inferred latches in synthesis
        next_state = current_state;
        val_addr   = current_ptr;
        vec_addr   = 0;
        mac_en     = 0;
        mac_clr    = 0;
        mac_weight = 0;
        mac_act    = 0;
        done       = 0;

        case (current_state)
            IDLE: begin
                mac_clr = 1; // Keep MAC accumulator at 0 while resting
                if (start && nnz_count > 0) begin
                    next_state = FETCH_PTR;
                end
            end

            FETCH_PTR: begin
                // Ask BRAM for the Value and the Column Index.
                val_addr = current_ptr;

                // We must wait here. BRAM has a 1-cycle read latency.
                next_state = FETCH_VEC;
            end
```

```
 97
 98              FETCH_VEC: begin
 99                  // The col_data has now arrived from the BRAM.
100                  // We immediately use it as the address to fetch the input vector.
101                  vec_addr = col_data[ADDR_WIDTH-1:0];
102                  next_state = COMPUTE;
103              end
104
105              COMPUTE: begin
106                  // All data has arrived. Pass it to the MAC and enable it.
107                  mac_weight = val_data;
108                  mac_act    = vec_data;
109                  mac_en     = 1;
110
111                  // Check if we are done with the array
112                  if (current_ptr == nnz_count - 1) begin
113                      next_state = DONE_STATE;
114                  end else begin
115                      next_state = FETCH_PTR; // Loop back for next element
116                  end
117              end
118
119              DONE_STATE: begin
120                  done = 1;
121                  // Wait for the testbench/CPU to drop the start signal
122                  if (!start) begin
123                      next_state = IDLE;
124                  end
125              end
126
127              default: next_state = IDLE;
128          endcase
129      end
130
131  endmodule
```

Line-by-line explanation

Declarations (lines 3–33). The module is parameterised by `ADDR_WIDTH` and `DATA_WIDTH`. Inputs `clk`, `rst_n`, `start` and `nnz_count` come from the host; `val_data`, `col_data` and `vec_data` are the registered BRAM outputs. Outputs `val_addr` and `vec_addr` address the BRAMs; `mac_en`, `mac_clr`, `mac_weight` and `mac_act` drive the MAC; `done` is the status flag.

State type (lines 36–42). `typedef enum logic [2:0]` declares five states encoded by default as 3'd0 (IDLE), 3'd1 (FETCH_PTR), 3'd2 (FETCH_VEC), 3'd3 (COMPUTE) and 3'd4 (DONE_STATE); `current_state` and `next_state` are the state register and its next value, and `current_ptr` (line 47) is the index of the non-zero being processed.

Sequential block (lines 52–66). On the falling edge of `rst_n` the state register returns to IDLE and the pointer to 0 (asynchronous reset). Otherwise at every rising edge `current_state` takes `next_state`; the pointer is incremented in the cycle that ends COMPUTE (`current_state == COMPUTE`) and cleared while the FSM is in IDLE. The pointer is therefore updated only after a MAC has consumed the corresponding operands.

Combinational block (lines 71–129). The default assignments at lines 73–80 (`next_state = current_state`, `val_addr = current_ptr`, all other outputs 0) guarantee that every output is assigned on every path, which prevents latch inference in synthesis. Notably `val_addr` equals `current_ptr` in *every* state, so the value and column-index banks are continuously reading the current non-zero. The state-specific behaviour is:

- **IDLE** asserts `mac_clr` so the accumulator rests at zero, and moves to FETCH_PTR when `start` && `nnz_count` > 0 (lines 83–88).
- **FETCH_PTR** presents `val_addr = current_ptr` and waits one cycle for the BRAM latency (lines 90–96).
- **FETCH_VEC** uses the now-valid `col_data` as the address of `bank_vector`: `vec_addr = col_data[ADDR_WIDTH-1:0]` (lines 98–103).
- **COMPUTE** forwards `val_data` and `vec_data` to `mac_weight` and `mac_act`, asserts `mac_en`, and decides between DONE_STATE (`current_ptr == nnz_count - 1`) and FETCH_PTR (lines 105–117).
- **DONE_STATE** asserts `done` and waits for `!start` before returning to IDLE (lines 119–125); the `default` branch returns to IDLE from any unused encoding.

Table 3.5 States of the five-state sparsity-aware controller

State	Encoding	Outputs asserted	Next state	Cycles
IDLE	000	`mac_clr` = 1	FETCH_PTR if `start` && `nnz_count` > 0; else IDLE	1 (while waiting)
FETCH_PTR	001	`val_addr` = ptr	FETCH_VEC	1
FETCH_VEC	010	`vec_addr` = `col_data`[5:0]	COMPUTE	1
COMPUTE	011	`mac_en` = 1, `mac_weight` = `val_data`, `mac_act` = `vec_data`	DONE_STATE if ptr = nnz − 1; else FETCH_PTR	1
DONE_STATE	100	`done` = 1	IDLE when `start` = 0; else DONE_STATE	≥ 1

Table 3.5 summarises the five states. Each stored non-zero traverses FETCH_PTR, FETCH_VEC and COMPUTE once, which is the origin of the three-cycles-per-non-zero rule derived in Section 3.11.

Multi-row CSR extension: the six-state scaling framework

The implemented core evaluates one compressed row per invocation. Extending it to a full matrix is a controlled evolution of the same schedule rather than a new design. The scaling framework adds a `row_ptr` BRAM bank, a result BRAM bank, a row counter and a six-state FSM:

1. **IDLE** – wait for `start`; clear the row counter and accumulator.
2. **FETCH_ROW_PTR** – read `row_ptr[r + 1]` (the start pointer is the previous end); form the row length; an empty row branches directly to WRITEBACK with a zero result.
3. **STREAM_CSR** – the existing FETCH_PTR/FETCH_VEC pair: issue the value/column address, wait the BRAM latency, issue the vector address.

4. **MAC_ACCUM** – the existing COMPUTE state: enable the MAC and increment the pointer; repeat STREAM_CSR/MAC_ACCUM until the pointer reaches `row_ptr[r + 1]`.
5. **WRITEBACK** – write the accumulator into the result bank at address *r*, pulse `clr_acc` and increment *r*; continue with FETCH_ROW_PTR or finish when *r* = `row_count`.
6. **DONE** – assert `done` and wait for `start` to fall, exactly as DONE_STATE does today.

Because STREAM_CSR and MAC_ACCUM are the present FETCH_PTR/FETCH_VEC and COMPUTE states, the datapath, the BRAM banks and the three-cycle inner schedule are reused unchanged; only the outer loop is new. Listing 3.4 gives the intended state skeleton.

Listing 3.4 Conceptual skeleton of the six-state multi-row CSR controller (scaling framework; not part of the synthesised five-state core)

```
typedef enum logic [2:0] {
    IDLE, FETCH_ROW_PTR, STREAM_CSR, MAC_ACCUM, WRITEBACK, DONE
} mstate_t;

always_comb begin
    next_state = state;  wr_en = 0;  mac_clr = 0;  mac_en = 0;  done = 0;
    case (state)
        IDLE:          if (start && row_count > 0) next_state = FETCH_ROW_PTR;
        FETCH_ROW_PTR: // row_ptr[r+1] arrives one cycle after the read
                       if (row_end == ptr) next_state = WRITEBACK;   // empty row
                       else                next_state = STREAM_CSR;
        STREAM_CSR:    next_state = MAC_ACCUM;       // value/col read, then vector read
        MAC_ACCUM: begin
            mac_en = 1;
            next_state = (ptr + 1 == row_end) ? WRITEBACK : STREAM_CSR;
        end
        WRITEBACK: begin
            wr_en = 1;  mac_clr = 1;                 // y[r] <= acc; acc <= 0
            next_state = (r + 1 == row_count) ? DONE : FETCH_ROW_PTR;
        end
        DONE:          begin done = 1; if (!start) next_state = IDLE; end
    endcase
end
```

3.11 BRAM latency and operand alignment

The schedule of the controller is dictated by two facts: every BRAM read has a latency of one clock cycle (the data appear on `douta` after the next rising edge), and the vector address depends on data read from a BRAM (the column index). Consider one stored non-zero with pointer *p*, and let the cycles of its processing be numbered 1, 2, 3.

- **Cycle 1 – FETCH_PTR.** `val_addr` = *p* is presented to `bank_values` and `bank_col_idx`. The data are not yet available; at the rising edge that ends the cycle both banks latch `mem[p]` into their output registers.
- **Cycle 2 – FETCH_VEC.** `val_data` = weight and `col_data` = column index are now valid. The controller forms `vec_addr` = `col_data[5:0]` combinationally and presents it to `bank_vector`; at the end of the cycle that bank latches *x*[`col_data`]

into `vec_data`. `val_data` remains stable because `val_addr` still equals *p* and the bank re-reads the same location.

- **Cycle 3 – COMPUTE.** `val_data` and `vec_data` are both valid and are forwarded to the MAC; `mac_en` = 1 so the product is added into the accumulator at the edge that ends the cycle, and the pointer increments at the same edge.

The two reads are *serially dependent*: `vec_addr` cannot be generated until `col_data` exists, and `col_data` cannot exist until one edge after `val_addr` is applied. Two registered BRAM reads in series therefore need two edges before the operand pair is complete, and the third cycle performs the arithmetic. The alignment is shown below; the symbol ▲ marks the clock edge that ends each cycle.

```
cycle            |   1: FETCH_PTR    |   2: FETCH_VEC       |   3: COMPUTE
-----------------+-------------------+----------------------+-----------------------
val_addr         |   p               |   p                  |   p
bank_values/col  |   read p  ... ▲   |   douta = mem[p]     |   douta = mem[p]
col_data         |   (old)           |   = col_idx[p]       |   = col_idx[p]
vec_addr         |   0 (default)     |   col_idx[p][5:0]    |   0 (default)
bank_vector      |   --              |   read x[col] ... ▲  |   douta = x[col]
mac_en / acc     |   0               |   0                  |   1 :  acc += w * x  ▲
current_ptr      |   p               |   p                  |   p  ->  p + 1  ▲
```

Latency formula. With NNZ stored non-zeros, `start` sampled in IDLE (1 cycle), three cycles per non-zero and one cycle in DONE_STATE in which `done` and the final result are visible, the execution latency in clock cycles is

$$T_{cycles} = 1 + 3 \cdot NNZ + 1 = 3 \cdot NNZ + 2 \tag{3.13}$$

For NNZ = 2 (the directed test) T = 8 cycles = 80 ns at 100 MHz; for the example of Section 5.9, NNZ = 410, T = 1,232 cycles. In general the throughput of the inner loop is one non-zero per three cycles, i.e. $f/3$ MAC operations per second at clock frequency f.

Scheduling headroom. The three-cycle schedule is the simplest correct schedule and is the one implemented and characterised. Because val_addr already advances to the next pointer at the edge that ends COMPUTE, FETCH_PTR of non-zero k + 1 can be overlapped with COMPUTE of non-zero k by pre-incrementing the pointer, which reduces the loop to two cycles per non-zero (T = 2 NNZ + 3, the extra cycle being the initial fetch); and a fully pipelined version, with the address generator, the vector-address stage and the MAC in three overlapped stages, accepts one non-zero per cycle ($T \approx$ NNZ + 4, one fill latency of three cycles plus the done cycle). These are the natural next steps once the system-level integration is complete.

3.12 INT8 multiply-accumulate datapath

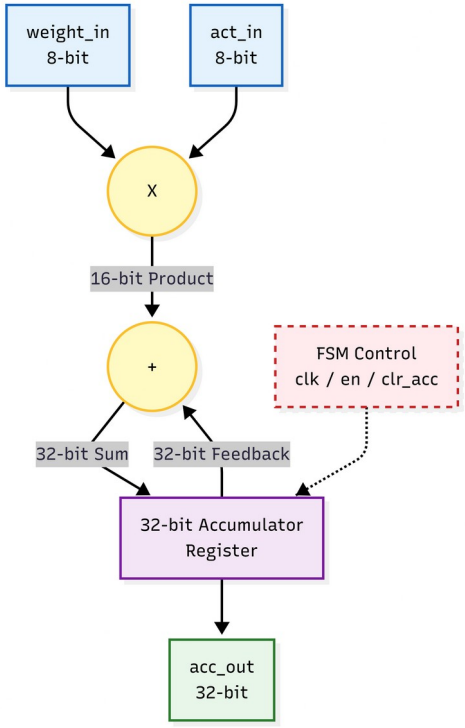


Figure 3.5 Datapath micro-architecture of the MAC unit (from the iSES paper): 8-bit signed multiplier, 16-bit product, 32-bit accumulator register with feedback

Figure 3.5 shows the datapath. Two signed 8-bit operands feed a multiplier that produces a 16-bit product; the product is sign-extended to 32 bits and added to the 32-bit accumulator register's feedback value; an FSM-controlled enable and clear determine whether the register loads the sum, holds, or clears. Listing 3.5 shows the complete module.

Listing 3.5 mac_unit.sv – combinational signed multiplier with enable-controlled 32-bit accumulator

```
`timescale 1ns / 1ps

module mac_unit (
    input  logic signed [7:0]  weight_in,
    input  logic signed [7:0]  act_in,
    input  logic               clk,
    input  logic               rst_n,
    input  logic               clr_acc,
    input  logic               en,
    output logic signed [31:0] acc_out
);

    // Combinational logic for the product
    logic signed [15:0] product;

    // SystemVerilog always_comb for pure combinational logic
    always_comb begin
        product = weight_in * act_in;
    end

    // Sequential logic for the accumulator using always_ff
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            acc_out <= 32'sd0;
        end else if (clr_acc) begin
            acc_out <= 32'sd0;
        end else if (en) begin
            acc_out <= acc_out + 32'(product); // Explicit casting for safety
        end
    end
```

```
31
32  endmodule
```

Explanation

- `weight_in, act_in [7:0] (signed)` – INT8 operands from the controller.
- `clk, rst_n` – clock and active-low asynchronous reset (the accumulator clears to zero when `rst_n` falls).
- `clr_acc` – synchronous clear: when 1 at a rising edge the accumulator is set to zero. The controller asserts it throughout IDLE, so every invocation starts from zero.
- `en` – accumulate enable: when 1 (and `clr_acc` = 0) the product is added at the rising edge; when 0 the register holds. The controller raises it only in COMPUTE, so cycles with no valid operand pair cannot alter the result.
- `acc_out [31:0] (signed)` – the accumulator, driven straight to `final_result`.
- `product [15:0] (signed)` – internal combinational product.

Combinational product. The `always_comb` block computes `product = weight_in * act_in`. Because the operands are signed and the destination is 16 bits, the result is the exact two's-complement product (Section 2.3); there is no register between the operands and the product, so the multiplier is purely combinational and its delay lies on the path from the BRAM output registers (through the controller's operand gating) to the accumulator.

Enable-controlled accumulation. The `always_ff` block has the priority order *reset* > *clear* > *enable*: on `rst_n` = 0 or `clr_acc` = 1 the register is zeroed, on `en` = 1 it loads `acc_out + 32'(product)`, and otherwise it keeps its value. In the target fabric this is a bank of 32 FDCE flip-flops (D flip-flop with clock enable and asynchronous clear), the synchronous clear folded into the data path and the clock-enable pin driven by the OR of `en` and `clr_acc`, so that holding the accumulator costs no extra logic and suppresses switching in the register when no operand is valid. The addition uses the slice carry chain.

3.13 Sparse accumulation and result interface

The result interface of the implemented core is the pair (`final_result`, `done`). `final_result` is the live output of the 32-bit accumulator. It is *not* a latched output: it takes the value 0 while the FSM rests in IDLE (the clear is asserted), changes after each COMPUTE, and is final in DONE_STATE. The host therefore samples it only when `done` = 1. Because the accumulator is cleared in IDLE and is not cleared in DONE_STATE, the value remains readable for the whole time `done` is asserted, i.e. until the host lowers `start` and the FSM returns to IDLE.

The accumulation law for one invocation with non-zeros (w_k, c_k), $k = 0, \ldots, \mathrm{NNZ}-1$, is

$$acc_0 = 0, \quad acc_{k+1} = acc_k + w_k \cdot x[c_k], \quad \texttt{final_result} = acc_{NNZ} \tag{3.14}$$

with each step realised by one COMPUTE cycle. This is exactly the row dot product of Section 3.4 for the row stored in the banks.

Multi-row write-back scaling. For a full matrix the accumulator is the row accumulator of a larger loop. At the end of each row the WRITEBACK state stores the accumulator into a result BRAM (address = row index), pulses `clr_acc` so that the next row starts from zero, and increments the row counter; `row_ptr` differences supply each row's length. The result vector *y* then resides in an on-chip bank that the host reads through the AXI BRAM controller, and a single `done` marks the end of the whole matrix. Counting two cycles of row-pointer fetch and one write-back cycle per row (Section 3.10), the latency of the extension is

$$T_{multi} = 1 + \sum_{r=0}^{M-1} (2 + 3n_r + 1) + 1 = 3 \cdot NNZ + 3M + 2 \qquad (3.15)$$

where n_r is the number of non-zeros of row r. For the 4 × 4 example (NNZ = 6, M = 4) this evaluates to 3·6 + 3·4 + 2 = 32 cycles; an empty row costs only its three overhead cycles. This is a design estimate for the scaling framework, to be confirmed by simulation of the extended controller.

3.14 Completion, reset, and repeated invocation

Reset (`rst_n`). The controller and the MAC use an active-low asynchronous reset (`always_ff @(posedge clk or negedge rst_n)`). Asserting `rst_n` returns the FSM to IDLE, sets the pointer to 0 and zeros the accumulator immediately, regardless of the clock. Releasing it is recommended to be synchronised to `clk` in the integrated system (the processor-system reset block of Section 3.21 provides this). The BRAM banks are not reset: their contents persist, which is what allows weights to be reused across invocations.

Clear (`clr_acc`). The controller asserts `mac_clr` in every cycle spent in IDLE, so the accumulator is zero on entry to FETCH_PTR without any explicit initialisation by the host. It is a *synchronous* clear, so it acts at a clock edge and allows the FSM to clear the accumulator at a row boundary without a hard reset of the system (the same signal is used by WRITEBACK in the multi-row extension).

`start` / `done` handshake. The protocol is a four-phase level handshake:

1. The host preloads the banks and sets `nnz_count`.
2. The host raises `start`. The FSM leaves IDLE at the next clock edge (if `nnz_count` > 0).
3. The FSM runs 3·NNZ cycles and enters DONE_STATE; `done` rises and `final_result` is valid.
4. The host reads the result and lowers `start`. In DONE_STATE the FSM sees `start` = 0 and returns to IDLE; `done` falls.

DONE_STATE retention. While `start` stays high the FSM remains in DONE_STATE and `done` and `final_result` remain stable. This retention makes the result safe to read by a slow host (for example over an AXI register) without a latch or FIFO, and prevents an accidental second run: re-execution requires an explicit start-low/start-high cycle. A guard is also provided at the input: `start` with `nnz_count` = 0 is ignored, so an empty invocation cannot underflow the termination compare `current_ptr == nnz_count - 1`. `start` is not examined in FETCH_PTR, FETCH_VEC or COMPUTE, so lowering it mid-run does not abort the computation.

Repeated invocation. Because the banks persist, a second invocation on the same weights but a new input vector only rewrites `bank_vector` through Port B and repeats the handshake, which is the pattern of repeated inference with a fixed (pruned) layer.

3.15 Module organization and verification hierarchy

The design comprises four RTL modules and two testbenches in six source files. The hierarchy is `sparse_cim_top` → {`sparse_cim_controller`, `mac_unit`, `bram_bank` × 3}; `tb_sparse_cim_top` instantiates the top level and `tb_mac_unit` instantiates the MAC alone. Table 3.6 maps modules to files and roles.

Table 3.6 RTL modules and testbenches

Module	Source file	Role	Instances
`sparse_cim_top`	sparse_cim_top.sv	Top level; wires controller, MAC and three BRAM banks; exposes start/done, `nnz_count`, result and Port B preload ports	1 (DUT of `tb_sparse_cim_top`)
`sparse_cim_controller`	sparse_cim_controller.sv	Five-state zero-skipping FSM; address and MAC control	1
`mac_unit`	mac_unit.sv	Signed INT8 multiplier and 32-bit accumulator	1
`bram_bank`	bram_bank.sv	Parameterised dual-port block-RAM bank	3 (values, col_idx, vector)
`tb_mac_unit`	tb_mac_unit.sv	Directed unit test of the arithmetic datapath	Simulation only
`tb_sparse_cim_top`	tb_sparse_cim_top.sv	Top-level directed test: preload, run, observe `done` and result	Simulation only

The verification hierarchy is bottom-up. The MAC is first tested in isolation, where the arithmetic is observable directly at `acc_out`; the top-level testbench then validates the integrated behaviour (memory preload, indirect addressing, FSM sequencing and handshake) on a known data set. Each testbench is self-contained and uses implicit port connection (`.*`), so adding a port to a DUT requires no edit to its testbench provided the names match.

3.16 Verification strategy and acceptance checks

Listing 3.6 gives the MAC unit testbench in full. It generates a 100 MHz clock (`always #5 clk = ~clk`), applies a reset for 15 ns, and walks the MAC through five scenarios.

Listing 3.6 tb_mac_unit.sv – directed testbench for the MAC unit

```
`timescale 1ns / 1ps

module tb_mac_unit;

    // All signals use the 'logic' type in SystemVerilog
    logic signed [7:0]  weight_in;
    logic signed [7:0]  act_in;
    logic               clk;
    logic               rst_n;
    logic               clr_acc;
    logic               en;
    logic signed [31:0] acc_out;

    // Instantiate Unit Under Test (UUT) using implicit port connections
    mac_unit uut (.*);

    // Clock generation (100 MHz, 10ns period)
    always #5 clk = ~clk;

    initial begin
        // Initialize Signals
        clk       = 0;
        rst_n     = 0;
        clr_acc   = 0;
        en        = 0;
        weight_in = 8'sd0;
        act_in    = 8'sd0;

        // Apply Reset
        #15 rst_n = 1;

        // Test 1: First MAC operation (5 * 3 = 15)
        @(posedge clk);
        en        = 1;
        weight_in = 8'sd5;
        act_in    = 8'sd3;

        // Test 2: Second MAC accumulation (15 + (-4 * 2) = 7)
        @(posedge clk);
        weight_in = -8'sd4;
        act_in    = 8'sd2;

        // Test 3: Disable enable (Sparsity condition / skipped cycle)
        @(posedge clk);
        en        = 0;
        weight_in = 8'sd10; // Ignored because en = 0
        act_in    = 8'sd10;

        // Test 4: Clear accumulator
        @(posedge clk);
        clr_acc   = 1;

        @(posedge clk);
        clr_acc   = 0;

        #20;
        $finish;
    end

endmodule
```

Stimulus phases of tb_mac_unit

- `Initialisation and reset (lines 22–30)` – all inputs are driven to 0 and `rst_n` is held low for 15 ns, so the accumulator is cleared by the asynchronous reset before any operand is applied.

- `Test 1 – first MAC (lines 32–36)` – `en` = 1, 5 × 3: the accumulator should load 15.
- `Test 2 – signed accumulation (lines 38–41)` – −4 × 2 = −8 is added: 15 + (−8) = 7, verifying negative-operand multiplication and sign-extended accumulation.
- `Test 3 – enable low (lines 43–47)` – `en` = 0 while the operands are set to 10 × 10: the value must stay 7. This is the “skipped cycle” that models a zero element being bypassed.
- `Test 4 – clear (lines 49–54)` – `clr_acc` = 1 for one cycle: the accumulator returns to 0 even though `en` is 0, then `clr_acc` is released.
- `Termination (lines 56–57)` – 20 ns of idle time and `$finish`.

The intended accumulator sequence is therefore 0 → 15 → 7 → 7 → 0, with each step taking effect at the clock edge following the stimulus change.

Listing 3.7 gives the top-level testbench.

Listing 3.7 tb_sparse_cim_top.sv – top-level directed testbench with BRAM preload and start/done handshake

```
`timescale 1ns / 1ps

module tb_sparse_cim_top;

    // Parameters
    localparam int DATA_WIDTH = 8;
    localparam int ADDR_WIDTH = 6;

    // Signals
    logic clk;
    logic rst_n;
    logic start;
    logic [ADDR_WIDTH-1:0] nnz_count;
    logic signed [31:0] final_result;
    logic done;

    // BRAM Preload signals
    logic web_val, web_col, web_vec;
    logic [ADDR_WIDTH-1:0] addrb;
    logic [DATA_WIDTH-1:0] dinb_val, dinb_col, dinb_vec;

    // Instantiate Top Module
    sparse_cim_top #(
        .DATA_WIDTH(DATA_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    ) uut (.*);

    // Clock generation
    always #5 clk = ~clk;

    initial begin
        // Initialize everything to 0
        clk = 0;
        rst_n = 0;
        start = 0;
        nnz_count = 0;
        web_val = 0; web_col = 0; web_vec = 0;
        addrb = 0;
        dinb_val = 0; dinb_col = 0; dinb_vec = 0;

        // Apply reset
        #20 rst_n = 1;

        // ========================================
        // PHASE 1: Preload the BRAMs via Port B
        // ========================================
```

```
47          @(posedge clk);
48          web_val = 1; web_col = 1; web_vec = 1;
49
50          // Write 1st Non-Zero Element (Weight = 5, Col = 0, Vector[0] = 3)
51          addrb = 0;
52          dinb_val = 8'sd5;
53          dinb_col = 8'd0;
54          dinb_vec = 8'sd3;
55          @(posedge clk);
56
57          // Write 2nd Non-Zero Element (Weight = -4, Col = 2, Vector[2] = 2)
58          // Note: We are writing the vector data to address 2!
59          addrb = 1;              // The CSR arrays are packed tightly (addr 1)
60          dinb_val = -8'sd4;
61          dinb_col = 8'd2;        // Points to index 2 in the vector
62
63          // Temporarily disable vector write to jump to address 2 safely
64          web_vec = 0;
65          @(posedge clk);
66
67          // Now write the vector data at address 2
68          web_val = 0; web_col = 0; web_vec = 1;
69          addrb = 2;
70          dinb_vec = 8'sd2;
71          @(posedge clk);
72
73          // Stop writing
74          web_vec = 0;
75
76          // ========================================
77          // PHASE 2: Start the Accelerator
78          // ========================================
79          #20;
80          @(posedge clk);
81          nnz_count = 2; // Tell it there are 2 non-zero elements
82          start = 1;
83
84          // Wait for the done signal from the FSM
85          wait(done == 1'b1);
86          @(posedge clk);
87          start = 0;
88
89          // Let it rest for a moment so we can see the final waveform
90          #50;
91          $finish;
92      end
93
94  endmodule
```

Stimulus phases of tb_sparse_cim_top

- `Initialisation (lines 32–39)` – clock, reset, `start`, `nnz_count` and all preload signals are set to 0; `rst_n` is released after 20 ns.
- `Phase 1 – preload through Port B (lines 44–74)` – at address 0 the testbench writes weight 5, column 0 and vector element $x[0] = 3$ into the three banks; at address 1 it writes weight −4 and column 2 (with the vector write enable temporarily low, so no vector word is stored at address 1); finally, with the value and column write enables low, it writes $x[2] = 2$ at address 2. The note in the source records the reason for the pause: the compressed arrays are packed from address 0, whereas the dense vector is indexed by column.
- `Phase 2 – run (lines 76–87)` – after 20 ns `nnz_count` = 2 and `start` = 1 are applied; `wait(done == 1'b1)` blocks until the FSM reaches DONE_STATE, then `start` is released one clock later.

- `Observation window (lines 89–91)` – 50 ns of simulation after the handshake so that the return to IDLE and the clearing of the accumulator are visible, then `$finish`.

Self-checking scoreboard extension. The directed testbenches expose the signals for waveform inspection and give a precise expected sequence. The natural extension, which the same structure supports, is a self-checking scoreboard that computes the expected value from the preloaded arrays with a software golden model and compares it with `final_result` when `done` rises. Together with driving stimulus from non-blocking assignments (so that the sampling order of stimulus and design at a clock edge is independent of simulator scheduling), randomised (`values`, `col_idx`, `vec`) tuples and a cycle-count assertion ($T = 3 \cdot \mathrm{NNZ} + 2$), this turns the directed tests into a regression suite. Listing 3.8 gives the intended scoreboard.

Listing 3.8 Self-checking scoreboard extension (proposed)

```
// Golden model: dot product of the preloaded compressed row (testbench shadow arrays)
function automatic int signed golden(input int n);
    int signed acc = 0;
    for (int k = 0; k < n; k++)
        acc += $signed(val_mem[k]) * $signed(vec_mem[col_mem[k][5:0]]);
    return acc;
endfunction

task automatic run_and_check(input int n);
    int cyc = 0;
    @(negedge clk);  nnz_count = n;  start = 1;      // drive away from the active edge
    while (!done) begin @(negedge clk); cyc++; end    // sample away from the active edge
    assert (final_result === golden(n))
        else $error("result %0d != expected %0d", final_result, golden(n));
    assert (cyc == 3*n + 1)                           // IDLE + 3n + DONE
        else $error("latency %0d != %0d", cyc, 3*n + 1);
    @(negedge clk);  start = 0;
endtask
```

Table 3.7 Acceptance checks for the Sparse-CIM core

ID	Check	Stimulus	Expected outcome	Status
V1	Signed multiply	5 × 3, then −4 × 2 (MAC TB)	acc: 15 → 7	Directed test
V2	Enable gating	`en` = 0 with operands 10, 10	acc holds 7	Directed test
V3	Synchronous clear	`clr_acc` = 1	acc = 0	Directed test
V4	BRAM preload	Port B writes, three banks	Operands stored at stated addresses	Directed test
V5	Indirect addressing	col_idx = {0, 2}, *x*[0] = 3, *x*[2] = 2	5(3) + (−4)(2) = 7	Waveform (Figure 5.1)
V6	FSM sequencing	`start` = 1, `nnz_count` = 2	FETCH_PTR → FETCH_VEC → COMPUTE ×2 → DONE_STATE	Waveform (Figure 5.1)
V7	Handshake and clear	Release `start`	`done` falls; accumulator cleared in IDLE	Waveform (Figure 5.1)
V8	Self-checking regression	Randomised CSR rows, golden model	Result and cycle count match	Roadmap (Listing 3.8)
V9	Multi-row sequencing	4 × 4 example, `row_ptr` = [0,2,3,4,6]	*y* = (13, 3, 12, 24)	Roadmap (Section 3.10)

3.17 Synthesis and resource optimization

Out-of-context (OOC) synthesis in Vivado synthesises the module as a stand-alone block with no I/O buffers, so the reported area and power describe the accelerator and not the I/O pads of the device [32]. Four aspects of the result are explained here and quantified in Section 5.5.

RAMB18E1 inference. Each `bram_bank` is 64 × 8 = 512 bits. The `ram_style = "block"` attribute and the registered-read coding style cause synthesis to infer one RAMB18E1 per bank, giving 3 RAMB18E1 in total (3/280 = 1.07 % of the device). A RAMB18E1 holds 18 Kb and can be configured as 2K × 9 [8], so each bank uses only about 3 % of the capacity of its primitive: raising `ADDR_WIDTH` to 11 (2,048 words) increases the capacity of every bank 32-fold at *no* additional block-RAM cost, and is the first step in scaling the tile size.

LUT-level arithmetic with 0 DSPs. The 8 × 8 signed multiplier, the 32-bit adder, the operand gating and the pointer logic are all implemented in slice LUTs and carry chains: the reported run uses 154 slice LUTs (154/53,200 = 0.29 %) and 0 DSP48E1 slices (0/220). The 32-bit accumulate uses the carry chain, and the 8 × 8 multiplier is the dominant consumer of LUTs, with the controller's output gating, the pointer incrementer and the end-of-loop comparator making up the remainder. A DSP48E1 offers a 25 × 18 multiplier [11], so using one for an 8 × 8 product would leave most of it idle while consuming a scarce DSP column site; moving the multiplier to LUTs removes that constraint, lets the placer put it beside the BRAM column, and allows dozens of PEs to coexist (Section 5.5). The mapping of arithmetic to LUT fabric is controlled at synthesis through the DSP-usage settings of Vivado [32]; recording the chosen setting in the project constraints makes the 0-DSP result reproducible.

FDCE register count. The 41 slice registers, all of type FDCE (D flip-flop with clock enable and asynchronous clear), decompose exactly into the 32-bit accumulator `acc_out`, the 6-bit pointer `current_ptr` and the 3-bit state register `current_state`: 32 + 6 + 3 = 41. The BRAM output registers are inside the RAMB18E1 primitives and are not counted as slice registers, and the 64 × 8 arrays require no flip-flops. The match between the architectural state of Sections 3.8–3.12 and the reported register count is a consistency check on the synthesised netlist.

Latch-free control. The default-assignment style of the combinational block ensures that synthesis infers no latches; the flip-flop count above is correspondingly free of extra storage elements.

3.18 Physical implementation methodology

After synthesis, implementation performs placement (assigning each LUT, flip-flop and RAM to a physical site), routing (selecting programmable interconnect between sites) and

timing closure. In the Zynq-7000 PL the block RAMs are arranged in vertical columns interleaved with the slice columns, and a net from a RAMB18E1 output to a slice LUT uses the general interconnect fabric; the delay of that connection is a function of the distance between the two sites. BRAM-to-logic locality is therefore a physical property that the placer controls rather than a logical property of the netlist.

For the Sparse-CIM PE this has three practical implications. (i) All fan-out from the BRAM outputs is small (the weight bus feeds the multiplier gating, the column bus feeds only the vector address, the vector bus feeds the multiplier gating), so the nets that matter for timing are few and short once the three banks are placed near the arithmetic. (ii) The use of LUT arithmetic rather than DSP48E1 leaves the placer free to choose the slice sites nearest to the RAM columns, whereas a DSP48E1 is fixed to a DSP column. (iii) For multiple PEs, each PE with its three banks should be kept in one clock region (for example with a Pblock constraint) so that its memory-to-compute connections remain local. Full place-and-route of the integrated design, with the block design clock and the AXI interconnect present, is part of the integration roadmap; the figures reported in this project are post-synthesis out-of-context values.

3.19 Timing constraints and clock interpretation

Two clock-related numbers appear in the project and must be interpreted separately. The **simulation clock** is 100 MHz (period 10 ns): the testbenches use `always #5 clk = ~clk`. It determines the time axis of the waveform (Figure 5.1) but it is not a timing constraint of the hardware. The **critical-path delay** of 10.32 ns is the post-synthesis estimate of the longest register-to-register path through the design, which, from the structure of the datapath, runs from the BRAM output registers through the operand gating and the combinational 8 × 8 multiplier and the 32-bit carry chain into the accumulator. The maximum clock frequency that this path supports is

$$F_{max} = 1 / T_{crit} = 1 / 10.32\ ns \approx 96.90\ MHz \tag{3.16}$$

and the setup slack at a target clock period T_{clk} is $T_{clk} - T_{crit}$ (ignoring clock uncertainty). Hence T_{clk} = 10.5 ns (95.2 MHz) gives a margin of +0.18 ns and T_{clk} = 11 ns (90.9 MHz) gives +0.68 ns; the 10 ns simulation period would correspond to −0.32 ns and indicates where the first pipelining step is needed. Listing 3.9 shows the constraint that expresses a 10.5 ns clock [33].

Listing 3.9 Clock constraint for implementation (Xilinx Design Constraints)

```
create_clock -name clk -period 10.500 [get_ports clk]
```

The path can be shortened without changing the architecture: registering the 16-bit product between the multiplier and the adder splits the path into two shorter ones (at the cost of one extra cycle of latency and 16 flip-flops), and registering the gated operands removes the

controller's multiplexing from the path. Either step moves F_{max} well beyond 100 MHz and keeps the compute rate at one non-zero per three cycles or better (Section 3.11). In the integrated SoC, the PL clock is generated by the processing system (FCLK_CLK0) at a frequency chosen to meet this constraint.

3.20 Custom IP packaging and proposed AXI4-Lite interface

To make the accelerator controllable by the ARM Cortex-A9, `sparse_cim_top` is to be wrapped as a custom IP core with an AXI4-Lite slave interface [35], using the Vivado IP packager [38]. The wrapper converts memory-mapped register accesses into the `start`, `nnz_count` and result signals of the core, and exposes the three BRAM banks to the processor through AXI BRAM controllers [39] on Port B. Table 3.8 gives the proposed register map. The base address is assigned by the Vivado address editor (typically 0x43C0_0000 for the first IP on the M_AXI_GP0 port).

Table 3.8 Proposed AXI4-Lite register map of the Sparse-CIM IP

Offset	Register	Access	Bits	Description
0x00	CTRL	R/W	[0] START; [1] SOFT_RESET	START drives `start`; the wrapper holds it until DONE and then releases it; SOFT_RESET drives `rst_n` low
0x04	STATUS	R/O	[0] DONE; [1] BUSY	DONE mirrors `done` (sticky until the next START); BUSY = START ∧ ¬DONE
0x08	NNZ_COUNT	R/W	[5:0] (6-bit instance)	Drives `nnz_count`: non-zeros per row for the single-row core, total non-zeros for the multi-row extension
0x0C	ROW_COUNT	R/W	[15:0]	Number of rows *M* for the multi-row extension; unused by the single-row core
0x10	RESULT	R/O	[31:0] signed	Drives from `final_result`; valid when STATUS.DONE = 1
Windows	BRAM banks	R/W	8-bit words	`bank_values`, `bank_col_idx`, `bank_vector` (and `row_ptr`/`y` banks in the extension) at separate AXI BRAM controller base addresses; written before START

The register semantics map one-to-one onto the handshake of Section 3.14. Because the RTL leaves DONE_STATE only after `start` falls, the wrapper either exposes START as a level that software clears, or (as proposed above) holds an internal start until DONE is observed and then clears it, making START a write-1 command. The DONE bit may also be routed to the PS interrupt input (IRQ_F2P) to avoid polling.

3.21 Processing-system and programmable-logic integration

Figure 3.6 shows the proposed integration in the Zynq-7000 SoC. The processing system's M_AXI_GP0 master port issues AXI transactions to an AXI interconnect, which routes register accesses to the AXI4-Lite slave wrapper and memory accesses to the AXI BRAM controller(s). The BRAM banks are true dual-port: Port B is connected to the controller for

streaming the CSR arrays and input vectors written by the ARM processor (or by DMA), and Port A is connected to the sparse-CIM core exactly as in the simulated design.

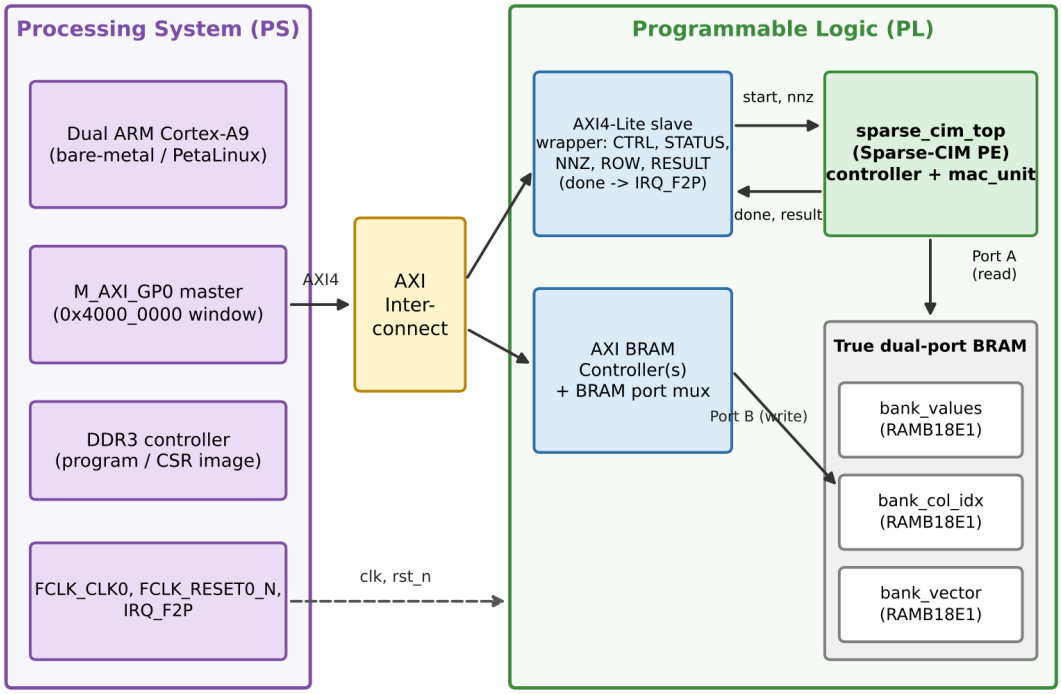


Figure 3.6 Proposed Zynq-7000 integration: ARM Cortex-A9 PS, AXI interconnect, AXI4-Lite wrapper, AXI BRAM controllers and the Sparse-CIM PE with dual-port banks

- **Clock.** The PS supplies FCLK_CLK0 to the AXI interconnect, the wrapper and the core, so there is a single synchronous domain and no clock-domain crossing inside the PE.
- **Reset.** A processor-system reset block turns FCLK_RESET0_N into the synchronised `rst_n` of the core.
- **Memory ownership.** Port B is written by the PS only while BUSY = 0; Port A is read only by the core. This preserves the conflict-free dual-port discipline of Section 3.9.
- **Data path.** Weights and CSR arrays reside in DDR3 under the PS; at layer load the processor (or a DMA engine) copies the arrays through the AXI BRAM controller into the on-chip banks, after which inference runs entirely on-chip.
- **Completion.** DONE is polled over AXI4-Lite or routed to IRQ_F2P.

The integration is built in Vivado IP Integrator [40]: the IP is added from the repository, connection automation instantiates the interconnect and reset block, and the Address Editor assigns the base addresses used by the driver of Section 3.23. Each added PE is an additional AXI slave with its own windows.

3.22 Bitstream export and embedded-software workflow

Moving from a validated block design to running software follows the sequence in Table 3.9. The hardware handoff is a **.xsa** (Xilinx Support Archive) file produced by Vivado's "Export Hardware" step with the bitstream included; it carries the PL configuration, the address map and the PS initialisation. Vitis consumes the XSA to create a platform and a domain (bare-metal *standalone* for the simplest flow, or PetaLinux/Linux when an operating system is needed) and generates the board-support package, including the header `xparameters.h` that defines the IP base addresses [41].

Table 3.9 Bitstream export and embedded-software workflow

Step	Tool	Input	Output	Purpose
1	Vivado IP Packager	RTL + AXI wrapper	Custom IP (`component.xml`)	Reusable IP core
2	IP Integrator	IP, PS7, interconnect, BRAM ctrl	Block design	SoC structure
3	Vivado	Block design	HDL wrapper, address map	Top-level netlist source
4	Synthesis, implementation, `write_bitstream`	HDL, constraints	`.bit`	PL configuration
5	Export Hardware	Implemented design	`.xsa` (with bitstream)	Hardware handoff
6	Vitis / PetaLinux	`.xsa`	Platform, BSP, `xparameters.h`	Software platform
7	Vitis build	C driver	`.elf`	Executable
8	Program and run	`.bit` + `.elf` (JTAG) or `BOOT.BIN` (SD)	Running system	Board execution

The same XSA flow is the route by which the project's synthesised core becomes a board-level demonstrator; the software of Step 7 is the subject of Section 3.23.

3.23 Host application control flow

The host application performs five actions: (i) convert the pruned INT8 matrix to CSR (offline or at start-up), (ii) preload the three banks over AXI, (iii) program `NNZ_COUNT` and set START, (iv) poll DONE with a timeout, (v) read RESULT and compare with a software reference. Listing 3.10 gives the driver pseudocode in C for the single-row core; the multi-row extension loops the same control flow over the row-pointer array.

Listing 3.10 C driver pseudocode for the Sparse-CIM IP (bare-metal, AXI4-Lite)

```
#include "xil_io.h"
#include "xparameters.h"          /* defines CIM_BASE, CIM_VAL_BASE, CIM_COL_BASE,
CIM_VEC_BASE */

#define REG_CTRL   0x00
#define REG_STATUS 0x04
#define REG_NNZ    0x08
#define REG_RESULT 0x10
#define START_BIT  0x1
#define DONE_BIT   0x1
#define TIMEOUT    1000000u

/* 1. Preload the compressed row and the dense vector into the BRAM banks (Port B) */
void cim_preload(const int8_t *val, const uint8_t *col, const int8_t *vec,
```

```
                  unsigned nnz, unsigned n)
{
    for (unsigned k = 0; k < nnz; k++) {
        Xil_Out32(CIM_VAL_BASE + 4*k, (uint8_t)val[k]);   /* two's-complement byte */
        Xil_Out32(CIM_COL_BASE + 4*k, col[k]);
    }
    for (unsigned j = 0; j < n; j++)
        Xil_Out32(CIM_VEC_BASE + 4*j, (uint8_t)vec[j]);
}

/* 2-4. Start, poll DONE, read the signed 32-bit result */
int cim_run(unsigned nnz, int32_t *result)
{
    Xil_Out32(CIM_BASE + REG_NNZ, nnz);
    Xil_Out32(CIM_BASE + REG_CTRL, START_BIT);             /* pulse / set START */

    unsigned t = TIMEOUT;
    while (!(Xil_In32(CIM_BASE + REG_STATUS) & DONE_BIT)) {
        if (--t == 0) return -1;                           /* hardware timeout */
    }
    *result = (int32_t)Xil_In32(CIM_BASE + REG_RESULT);    /* valid while DONE = 1 */
    Xil_Out32(CIM_BASE + REG_CTRL, 0);                     /* release START: FSM -> IDLE */
    return 0;
}

int main(void)
{
    const int8_t  val[2] = { 5, -4 };
    const uint8_t col[2] = { 0, 2 };
    const int8_t  vec[3] = { 3, 0, 2 };
    int32_t y;

    cim_preload(val, col, vec, 2, 3);
    if (cim_run(2, &y) == 0 && y == 7)                     /* expected: 5*3 + (-4)*2 */
        xil_printf("PASS: y = %d\r\n", (int)y);
    else
        xil_printf("FAIL\r\n");
    return 0;
}
```

Two details of the driver follow from the hardware. First, the byte written to a bank is the two's-complement representation of the INT8 value, so the cast `(uint8_t)` is a bit-for-bit copy, and the result is reinterpreted as `int32_t` on read. Second, START is released only after RESULT has been read, because the FSM retains DONE_STATE (and the result) until `start` falls.

3.24 Edge-inference applications and deployment scenarios

The engine targets the class of workloads in which a fixed, pruned weight matrix is applied to a stream of dense activation vectors under tight power and latency budgets. Four deployment families illustrate how the architecture would be used. Each assumes the roadmap integration of Sections 3.20–3.23; the quantitative results of this report are those of the core itself.

(a) Pruned CNNs, MLPs and Transformers

Fully connected and projection layers of pruned networks are SpMVM at batch size one. A tile of the pruned weight matrix, stored as CSR in the three banks, is loaded once per layer; the activation vector for each inference is written to `bank_vector`, and the PE returns one output element per row. Convolutional layers lowered to matrix form and the query/key/value and feed-forward projections of a small Transformer follow the same pattern. Larger layers are partitioned into tiles that fit the banks (for the 11-bit instance, 2,048 non-zeros per tile), processed by one PE in sequence or by several PEs in parallel (Section 5.5). Unstructured

pruning ratios of 70–90 % [2], [3] place these layers in the regime where Section 5.9 shows speedups of 3× or more over a dense scan.

(b) TinyML, keyword spotting and audio IoT

Keyword-spotting and sound-event detection run small MLPs or depthwise-separable CNNs on mel-frequency features every few tens of milliseconds. Their weights are small enough to reside permanently in on-chip block RAM, so the PE never accesses DRAM during inference; the batch-one, deterministic, always-on character of the workload matches the engine's cycle-exact latency $T = 3 \cdot \mathrm{NNZ} + 2$ (for a few thousand non-zeros, a few hundred microseconds at about 90 MHz, well within a 20–40 ms audio frame). Between frames the accumulator and FSM idle with no switching activity.

(c) Autonomous robotics, LiDAR point-cloud and SLAM graph processing

Mobile robots and drones combine learned perception with sparse numerical kernels. Voxelised LiDAR point clouds are inherently sparse (most voxels are empty), and sparse convolutions and graph-based pose optimisation in SLAM reduce to repeated sparse matrix–vector products with Jacobian or adjacency matrices of fixed structure [28], [27]. A fixed sparsity pattern is the ideal case for a CSR engine: the `row_ptr`/`col_idx` arrays are computed once and reused, and the deterministic latency of the FSM supports hard control-loop deadlines. Wider arithmetic (INT16 or FP32) would be needed for numerically sensitive solver steps, which is why the multi-precision scaling of Section 6.4.4 is part of the roadmap.

(d) Smart healthcare wearables, ECG and EEG biosignal classification

Biosignal classifiers operate on short windows of ECG or EEG samples and use compact 1-D CNNs or MLPs whose pruned weights fit in a few kilobytes. On-chip storage keeps patient data on the device, which supports privacy requirements, and a deterministic engine allows the worst-case latency of an arrhythmia alarm to be bounded analytically. Because the design uses only generic BRAM and LUT resources, the same RTL can be retargeted to lower-power FPGA families for wearable form factors, and the effective activity of the engine (the fraction of cycles in which the MAC is enabled) can be reduced further by gating BRAM enables outside the active states (Section 5.7).

CHAPTER 4
CONTRIBUTION OF STUDENT

This is an individual project. All work packages below were carried out by **Vemana Venkata Pujithram (230907016)** under the guidance of Dr. Rahul Ratnakumar.

4.1 Problem formulation, CSR data modelling, and near-memory architecture design (Vemana Venkata Pujithram — 230907016)

The first work package defined the engineering problem and the architecture. It comprised the literature study of sparse accelerators (EIE, SCNN, GraphLily, SERPENS, Cerberus and the INT8 FPGA and MAC references), the choice of CSR as the storage format on the basis of the storage and dataflow analysis of Section 3.4, and the decision to realise the in-memory-computing idea as BRAM-centric near-memory computing because analog bit-line access is not available on a commercial FPGA (Section 3.5). It also fixed the numerical design point (INT8 operands, 16-bit exact product, 32-bit accumulator), the three-bank memory organisation with dedicated read and write ports, and the controller schedule. Deliverables: the mathematical models of Sections 3.4 and 3.6, the architecture of Figure 3.3 and the cycle model of Section 3.11.

4.2 SystemVerilog RTL design of the BRAM banks, sparsity controller FSM, and INT8 MAC datapath (Vemana Venkata Pujithram — 230907016)

The second work package was the RTL implementation. It produced the four synthesisable modules: the parameterised dual-port `bram_bank` with the `ram_style = "block"` attribute, the five-state `sparse_cim_controller` with latch-free default assignments and the indirect vector-address path, the signed `mac_unit` with the reset/clear/enable priority and the sign-preserving 32-bit cast, and the top-level `sparse_cim_top` that integrates them with the backdoor Port B preload interface. Deliverables: the four source files listed in Chapter 3, together with the scaling framework (six-state multi-row controller) specified in Section 3.10.

4.3 Functional verification and directed testbench development (Vemana Venkata Pujithram — 230907016)

The third work package was verification. It comprised the MAC unit testbench (signed multiply, enable gating, clear), the top-level testbench (Port B preload with the vector-address pause, start/done handshake, wait-for-done synchronisation), the construction of the expected results $5(3) + (-4)(2) = 7$, and the interpretation of the Vivado Simulator waveform (Figure 5.1). It also defined the self-checking scoreboard and regression extension of Section 3.16 and the acceptance checks of Table 3.7.

4.4 Vivado synthesis, area/resource utilization, timing, and power analysis (Vemana Venkata Pujithram — 230907016)

The fourth work package was hardware characterisation. It comprised out-of-context synthesis on the Zynq-7000 `xc7z020clg484-1` device, extraction and interpretation of the utilisation report (154 LUTs, 41 FDCE flip-flops, 3 RAMB18E1, 0 DSP48E1), the critical-path delay (10.32 ns), the vectorless power estimate (4.502 W), the throughput and speedup models of Section 5.9, and the comparison with published accelerators. The results were consolidated into the IEEE iSES conference paper and into this report.

CHAPTER 5
RESULT ANALYSIS

5.1 Functional simulation results

Figure 5.1 shows the behavioural simulation of `tb_sparse_cim_top` in the Vivado Simulator. The waveform shows, from left to right, the reset release at 20 ns, the Port B preload (the `web_val`, `web_col` and `web_vec` pulses between 25 and 55 ns, with `addrb` stepping 0, 1, 2), the application of `nnz_count` = 2 and `start` = 1 at 75 ns, the accumulation of `final_result` from 0 to 15 (at 115 ns) and to 7 (at 145 ns), the two-cycle assertion of `done` from 145 to 165 ns, and the return of `final_result` to 0 at 175 ns when the FSM has returned to IDLE and cleared the accumulator.

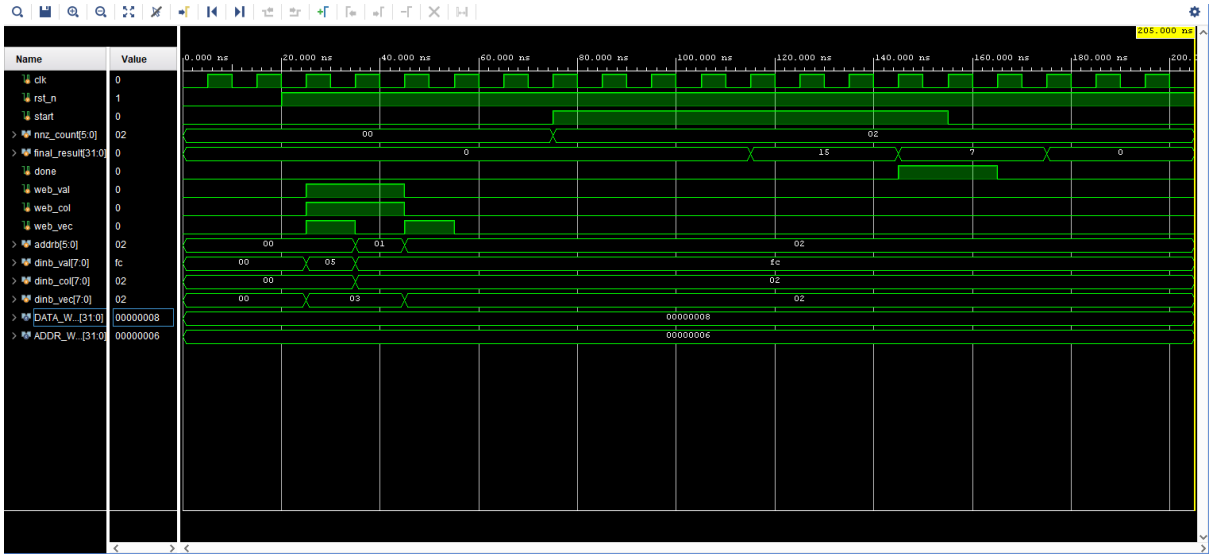


Figure 5.1 Behavioural simulation of the top module (from the iSES paper): BRAM preload, FSM zero-skipping sequence and accumulation 0 → 15 → 7

Table 5.1 reconstructs the same run cycle by cycle from the semantics of the RTL (a cycle-accurate reference model of the controller, BRAM read latency and MAC), with *t* measured from the 75 ns start of the testbench. Its `final_result` column reproduces the 0 → 15 → 7 → 0 sequence of the waveform at 115, 145 and 175 ns, which cross-checks the reconstruction.

Table 5.1 Cycle-by-cycle simulation values for the directed test (5 × 3 + (−4) × 2); *t* is the start time of each 10 ns cycle

Cyc.	*t* (ns)	FSM state	ptr	val	col	vaddr	vdata	en	prod.	result	done
0	75	IDLE	0	–	–	–	–	0	–	0	0
1	85	FETCH_PTR	0	–	–	–	–	0	–	0	0
2	95	FETCH_VEC	0	5	0	0	–	0	–	0	0
3	105	COMPUTE	0	5	0	–	3	1	15	0	0
4	115	FETCH_PTR	1	–	–	–	–	0	–	15	0
5	125	FETCH_VEC	1	-4	2	2	–	0	–	15	0
6	135	COMPUTE	1	-4	2	–	2	1	-8	15	0
7	145	DONE_STATE	2	–	–	–	–	0	–	7	1
8	155	DONE_STATE	2	–	–	–	–	0	–	7	1
9	165	IDLE	0	–	–	–	–	0	–	7	0
10	175	IDLE	0	–	–	–	–	0	–	0	0

Column key: val = `val_data`, col = `col_data`, vaddr = `vec_addr`, vdata = `vec_data`, en = `mac_en`, prod. = product, result = `final_result`; – means the signal carries no valid new value in that cycle.

The table can be read as follows. **Cycle 0 (IDLE)**: `start` and `nnz_count` = 2 are present; `mac_clr` keeps the accumulator at 0. **Cycles 1–3 (first non-zero)**: in FETCH_PTR the address 0 is applied; in FETCH_VEC the weight 5 and column 0 have arrived and `vec_addr` = 0 is issued; in COMPUTE `vec_data` = 3 has arrived, the product is 15 and `mac_en` = 1, so `final_result` becomes 15 at the end of the cycle. **Cycles 4–6 (second non-zero)**: pointer 1 is fetched; the weight is −4 (stored as 0xFC) and the column is 2, so `vec_addr` = 2 and `vec_data` = 2; the product is −8 and `final_result` becomes 7. **Cycle 7 (DONE_STATE)**: `done` = 1 and the result 7 is valid. **Cycle 8**: the testbench has lowered `start`, so the FSM leaves DONE_STATE at the end of the cycle; **cycle 9 (IDLE)** asserts the clear and **cycle 10** shows the accumulator at 0. The total from the first IDLE cycle to the first DONE_STATE cycle is 8 cycles, equal to 3·2 + 2.

For reference, the two code fragments that govern this run are re-listed below with their original line numbers: the COMPUTE branch of the controller, and the run phase of the testbench.

Listing 5.1 sparse_cim_controller.sv, lines 105–117: COMPUTE state (excerpt)

```
105             COMPUTE: begin
106                 // All data has arrived. Pass it to the MAC and enable it.
107                 mac_weight = val_data;
108                 mac_act    = vec_data;
109                 mac_en     = 1;
110
111                 // Check if we are done with the array
112                 if (current_ptr == nnz_count - 1) begin
113                     next_state = DONE_STATE;
114                 end else begin
115                     next_state = FETCH_PTR; // Loop back for next element
116                 end
117             end
```

Listing 5.2 tb_sparse_cim_top.sv, lines 76–91: run phase and observation window (excerpt)

```
76          // ========================================
77          // PHASE 2: Start the Accelerator
78          // ========================================
79          #20;
80          @(posedge clk);
81          nnz_count = 2; // Tell it there are 2 non-zero elements
82          start = 1;
83
84          // Wait for the done signal from the FSM
85          wait(done == 1'b1);
86          @(posedge clk);
87          start = 0;
88
89          // Let it rest for a moment so we can see the final waveform
90          #50;
91          $finish;
```

5.2 Interpretation of functional correctness and coverage

The simulation establishes five properties of the design. **(i) Preload correctness:** the operands written through Port B are the operands read by Port A, at the addresses the controller generates. **(ii) Indirect addressing:** the second non-zero has column index 2, and the vector element used is $x[2] = 2$, not the sequentially next element; a design that did not route `col_data` into `vec_addr` would produce a different sum. **(iii) Signed arithmetic:** the negative weight yields a negative product and the accumulator decreases from 15 to 7. **(iv) Control sequencing:** the FSM visits FETCH_PTR, FETCH_VEC and COMPUTE twice, once per non-zero, raises `done` immediately after the last accumulation, and clears the accumulator after `start` falls. **(v) Completion handshake:** `done` is held while `start` is high and released when it falls.

In terms of coverage, the directed tests exercise all five FSM states, the transitions IDLE → FETCH_PTR, FETCH_PTR → FETCH_VEC → COMPUTE, both COMPUTE exits (loop back to FETCH_PTR for the first non-zero, exit to DONE_STATE for the last) and DONE_STATE → IDLE; both signs of the product; all three arithmetic control states of the MAC (accumulate, hold, clear) and the reset. The coverage space that the planned scoreboard regression of Section 3.16 adds consists of randomised column indices and operand values, `nnz_count` from 1 to 63, multiple back-to-back invocations and the guard `nnz_count` = 0, together with automatic checking of the 3·NNZ + 2 latency.

5.3 Signed arithmetic and accumulator correctness

The test vector (5, 3) followed by (−4, 2) exercises the following behaviour of the datapath. The weight −4 is stored as the byte 0xFC (two's complement). The multiplier sees the signed value −4, and the 16-bit product with 2 is 0xFFF8 = −8. The size cast `32'(product)` sign-extends it to 0xFFFFFFF8, and the 32-bit addition 0x0000000F + 0xFFFFFFF8 = 0x00000007 (with the carry out of bit 31 discarded) equals 7. The accumulator therefore behaves as a two's-complement integer and no special handling of negative products is required.

Overflow bounds. The largest product magnitude is $|-128 \cdot -128| = 16{,}384 = 2^{14}$ and the 16-bit product, whose range is [−32,768, 32,767], never overflows. For the accumulator, after k accumulations $|\text{acc}| \le k \cdot 2^{14}$, which stays below 2^{31} while

$$k < 2^{31} / 2^{14} = 2^{17} = 131{,}072 \tag{5.1}$$

The 6-bit instance processes at most 63 non-zeros per invocation ($|\text{acc}| \le 1{,}032{,}192 < 2^{20}$), so the 32-bit register has more than 11 unused bits of headroom; even the 11-bit instance with 2,047 non-zeros ($|\text{acc}| \le 33.5$ million $< 2^{25}$) retains six. The 32-bit width, equal to the width of the result port, thus guarantees overflow-free accumulation for any layer tile addressable by the banks, at a cost of 32 flip-flops.

5.4 Storage footprint and compression efficiency

Using the expressions of Section 3.4 for a 64 × 64 INT8 tile ($b_v = 8$, $b_c = b_r = 6$), the dense footprint is 32,768 bits (4.00 KiB) independent of sparsity, whereas the CSR footprint with minimum-width indices is 29,452 bits (3.60 KiB) at 50 % sparsity (NNZ = 2048); 17,921 bits (2.19 KiB) at 70 % sparsity (NNZ = 1229); 6,325 bits (0.77 KiB) at 90 % sparsity (NNZ = 410); 3,390 bits (0.41 KiB) at 95 % sparsity (NNZ = 205). The corresponding compression ratios relative to dense are 1.1×, 1.8×, 5.2×, 9.7× respectively. Figure 5.2 compares the four formats; the COO footprint exceeds CSR at every sparsity because of its per-element row index, and the break-even of Section 3.4 is visible at 50 % sparsity, where COO is larger than dense and CSR is only 10 % smaller.

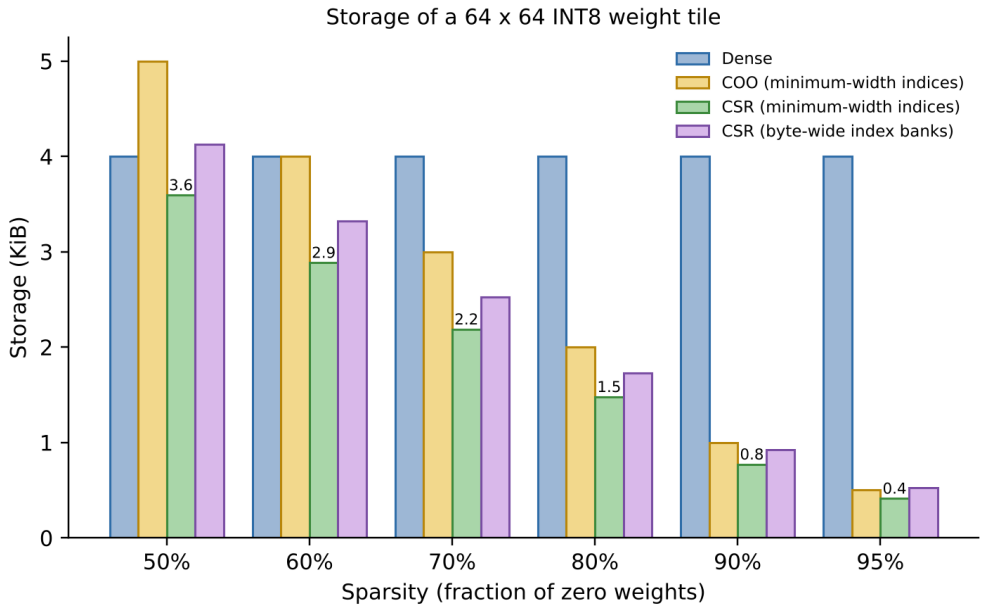


Figure 5.2 Storage of a 64 × 64 INT8 weight tile for Dense, COO and CSR formats versus sparsity

The byte-wide index banks of the implementation (`DATA_WIDTH` = 8 for the column index and for the pointer) cost between 15 % and 27 % more than the minimum-width bound, as the right-hand bar of each group shows. This is a deliberate simplification: the BRAM word is 8

bits wide in any case, since a RAMB18E1 is organised in 9-bit-wide words and the extra bits would be unused. The saving of the on-chip memory is therefore best expressed in *words read* (Table 3.3) rather than in minimum-width bits.

5.5 FPGA utilization and architectural bottlenecks

Table 5.2 reproduces the post-synthesis resource utilisation of the accelerator on the `xc7z020clg484-1` device (out-of-context synthesis), with percentages computed against the device inventory.

Table 5.2 FPGA resource utilisation of the Sparse-CIM core on the Zynq-7000 (xc7z020clg484-1)

Resource	Utilisation	Available	Utilisation (%)
Slice LUTs	154	53,200	0.29
Slice Registers (FDCE)	41	106,400	0.04
Block RAM (RAMB18E1)	3	280	1.07
DSP48E1 slices	0	220	0.00

The core occupies under 0.3 % of the LUTs, 0.04 % of the flip-flops and about 1 % of the block RAMs, and no DSP slices. The 41 registers decompose exactly as 32 (accumulator) + 6 (pointer) + 3 (state) (Section 3.17), and the 3 RAMB18E1 are the three operand banks.

Multi-PE parallel scaling

Because the engine is so small, the device can host many PEs, each with its own three banks. The number of PEs that fit is bounded by the scarcest resource:

- LUTs: $\lfloor 53200/154 \rfloor = 345$ PEs;
- flip-flops: $\lfloor 106400/41 \rfloor = 2595$ PEs;
- RAMB18E1 (one per bank, three per PE): $\lfloor 280/3 \rfloor = 93$ PEs;
- DSP48E1: not a constraint (0 per PE).

Block RAM is therefore the binding resource: 93 PEs would use 14,322 LUTs (26.9 %), 3,813 flip-flops (3.6 %) and 279 RAMB18E1 (99.6 %). Each bank uses only 512 of the 2,048 words (9-bit) of its primitive, so address-partitioned sharing of a primitive between the banks of several PEs is available if more PEs are wanted, at the cost of port arbitration; alternatively the same three primitives per PE can hold up to 32× larger tiles (`ADDR_WIDTH` = 11) at no area cost. Under ideal linear scaling (independent PEs on independent rows, ignoring routing congestion and clock degradation) the aggregate effective throughput would be about $93 \times 0.64 \approx 60$ GOPS-equivalent at 90 % sparsity; this is a projection, not a measurement.

The architectural bottlenecks of a single PE are, in order: (i) the 3-cycle-per-non-zero schedule, which limits one PE to $f/3$ MACs per second; (ii) the single MAC per PE; (iii) the combinational multiplier-plus-adder path that sets $T_{\text{crit}} = 10.32$ ns; and (iv) the single read port per bank on Port A. Each is addressed by a localised change (overlapped fetch, parallel MAC

lanes with banked indices, a product register, and bank replication respectively) that preserves the architecture.

5.6 Timing results and their scope

Table 5.3 Timing results and derived quantities (post-synthesis, out-of-context)

Quantity	Value	Remark
Critical path delay T_{crit}	10.32 ns	Longest register-to-register path (post-synthesis)
Maximum frequency F_{max}	≈ 96.90 MHz	1 / 10.32 ns
Simulation clock	100 MHz (10 ns)	Behavioural simulation only; not a constraint
Slack at T_{clk} = 10.5 ns	+0.18 ns	95.2 MHz target
Slack at T_{clk} = 10.0 ns	−0.32 ns	Would need the pipelining step below
Time per non-zero at F_{max}	30.96 ns	3 cycles × 10.32 ns
Non-zero rate at F_{max}	32.30 M non-zeros/s	F_{max} / 3

The reported delay is the post-synthesis figure for the out-of-context core: it includes the logic delay and the synthesis tool's estimate of net delay, and it does not include the placed-and-routed interconnect of a full system. It is therefore the appropriate number for comparing micro-architectures and for planning the clock constraint, and the implementation of the integrated design (Section 3.18) will refine it.

Pipeline-optimisation avenues. The 10.32 ns path comprises, in series, the BRAM clock-to-output delay, the controller's operand gating, the 8 × 8 LUT multiplier and the 32-bit carry-chain accumulate. Three independent register insertions shorten it: (a) a 16-bit product register after the multiplier, splitting multiply from accumulate; (b) registering `mac_weight`/`mac_act` after the gating; (c) enabling the optional output register of the RAMB18E1 [8], which reduces the clock-to-output delay at the cost of one extra read-latency cycle. Each adds one pipeline stage and a small number of flip-flops (16, 16, and 8 per bank respectively) and shifts the bottleneck to the next-longest stage; combined with the overlapped schedule of Section 3.11 the engine reaches one non-zero per cycle at a clock considerably above 100 MHz.

5.7 Estimated power and characterization analysis

Figure 5.3 shows the post-synthesis out-of-context power summary produced by Vivado, and Table 5.4 tabulates it. The estimate is *vectorless*: it is derived from the synthesised netlist and from the tool's default switching-activity assumptions, without simulation activity data [34]. The report itself marks the confidence level as *Low*, which is the expected classification for any pre-implementation, vectorless estimate.

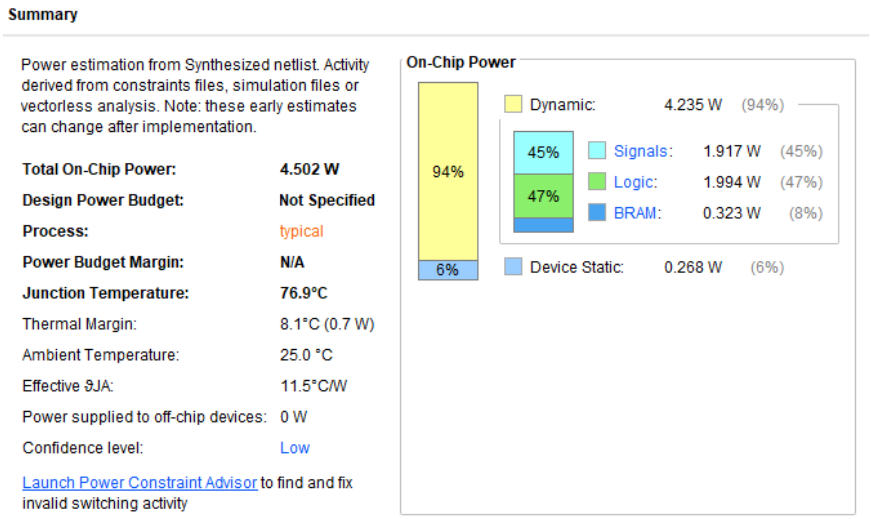


Figure 5.3 Post-synthesis out-of-context power estimate (from the iSES paper): total on-chip power 4.502 W

Table 5.4 Vectorless post-synthesis power estimate (xc7z020clg484-1, out-of-context)

Component	Power (W)	Share
Total on-chip power	**4.502**	100 %
Dynamic power	4.235	94 % of total
Signals	1.917	45 % of dynamic
Logic	1.994	47 % of dynamic
BRAM	0.323	8 % of dynamic
Device static power	0.268	6 % of total
Junction temperature / thermal margin	76.9 °C / 8.1 °C (0.7 W)	θ_{JA} = 11.5 °C/W
Confidence level	Low	Vectorless estimate

Reading the estimate constructively. A vectorless analysis assigns a default toggle rate and static probability to every net that has no simulated activity, so the result is an *envelope*: it bounds the power of a design whose nets are all assumed to be switching at the default rate, rather than the power of a specific inference. Under this model the distribution is informative. Signals (routing) and logic together account for 92 % of the dynamic power and the three RAMB18E1 only 8 % (0.323 W), which agrees with the architectural intent: the operand storage is cheap and the energy is spent where switching is assumed. The device static term, 0.268 W, is a property of the chosen silicon and is independent of the design.

The characterisation can be tightened in three ways that the existing RTL supports. First, **activity-annotated power**: exporting a SAIF or VCD file from the testbench simulation and reading it into the power analyser replaces default activity with the actual, mostly idle, toggling of an inference. Second, **activity-aware enables**: the MAC accumulator is already enable-gated; driving the Port A enable of each bank from the FSM state, instead of tying `ena` to 1 (Listing 3.1), suppresses BRAM reads in IDLE, DONE_STATE and the cycles that do not need a fetch. Third, **measurement** on a board (Section 5.8). The 4.502 W figure is

also recorded in the iSES paper and in the comparison of Section 5.9, where its vectorless nature is noted.

5.8 Hardware demonstration status and deployment boundary

The results of this report are RTL-level and post-synthesis results. The engine has been simulated (Section 5.1) and synthesised for the Zynq-7000 xc7z020clg484-1 (Sections 5.5–5.7). The step that remains is its deployment on a physical Zynq-7000 board, which proceeds exactly as specified in Sections 3.20–3.23: package the core with the AXI4-Lite wrapper, build the block design, export the XSA, run the driver of Listing 3.10 and compare the on-board result and cycle count with the simulation values of Table 5.1.

The boundary between the demonstrated and the roadmap items is therefore: **demonstrated** — correct signed accumulation with indirect addressing, the five-state schedule, the handshake, and the area/timing/power characterisation; **roadmap** — multi-row CSR sequencing, automatic checking, AXI integration and on-board measurement. When the board measurements are available, the figures of Chapter 5 gain three measured counterparts: the end-to-end latency (read from a hardware counter in the wrapper), the on-board power (from the board's power monitor) and the board-level correctness of repeated invocations.

5.9 Latency, throughput, speedup, and architectural comparison

Cycle-accurate latency. Section 3.11 shows that every stored non-zero costs three cycles (FETCH_PTR, FETCH_VEC, COMPUTE), because the one-cycle synchronous read of `bank_col_idx` must complete before `vec_addr` can be issued to `bank_vector`, whose own one-cycle read must complete before the MAC can operate. Adding the IDLE cycle in which `start` is sampled and the DONE_STATE cycle in which `done` is visible,

$$T_{cycles} = 1 + 3 \cdot NNZ + 1 \tag{5.2}$$

For the directed test, NNZ = 2 and T = 8, which Table 5.1 confirms (IDLE at cycle 0, DONE_STATE at cycle 7). The latency in time is $T_{\text{time}} = T_{\text{cycles}} / F$.

Speedup. Two baselines are meaningful. The first is a *dense scan on the same PE*, which visits all MN elements with the same three-cycle schedule: $T_{\text{dense},3} = 3MN + 2$, so

$$S_3 = (3MN + 2) / (3\ NNZ + 2) \approx 1 / (1 - s) \tag{5.3}$$

The second is an *ideal pipelined dense engine* that performs one MAC per cycle, $T_{\text{dense},1} = MN$, which is a demanding baseline because the sparse PE needs three cycles per non-zero: $S_1 = MN / (3\ \text{NNZ} + 2) \approx 1/(3(1 - s))$. The sparse PE therefore overtakes the ideal dense engine when sparsity exceeds about 67 % ($s > 2/3$), and the overlapped and pipelined schedules of Section 3.11 move this crossover to 50 % and below. Table 5.5 evaluates both for a 64 × 64 tile (MN = 4096) at F_{max} = 96.9 MHz, together with the latency and the effective (dense-equivalent) throughput $2MN \cdot F/T$.

Table 5.5 Latency, speedup and effective throughput versus sparsity (64 × 64 INT8 tile, F = 96.9 MHz)

Sparsity	NNZ	T (cycles)	Latency (µs)	Speedup vs dense scan (3-cycle)	Speedup vs ideal dense (1 MAC/cycle)	Eff. GOPS
50 %	2,048	6,146	63.43	2.00×	0.67×	0.129
60 %	1,638	4,916	50.73	2.50×	0.83×	0.161
70 %	1,229	3,689	38.07	3.33×	1.11×	0.215
80 %	819	2,459	25.38	5.00×	1.67×	0.323
90 %	410	1,232	12.71	9.98×	3.32×	0.644
95 %	205	617	6.37	19.92×	6.64×	1.287

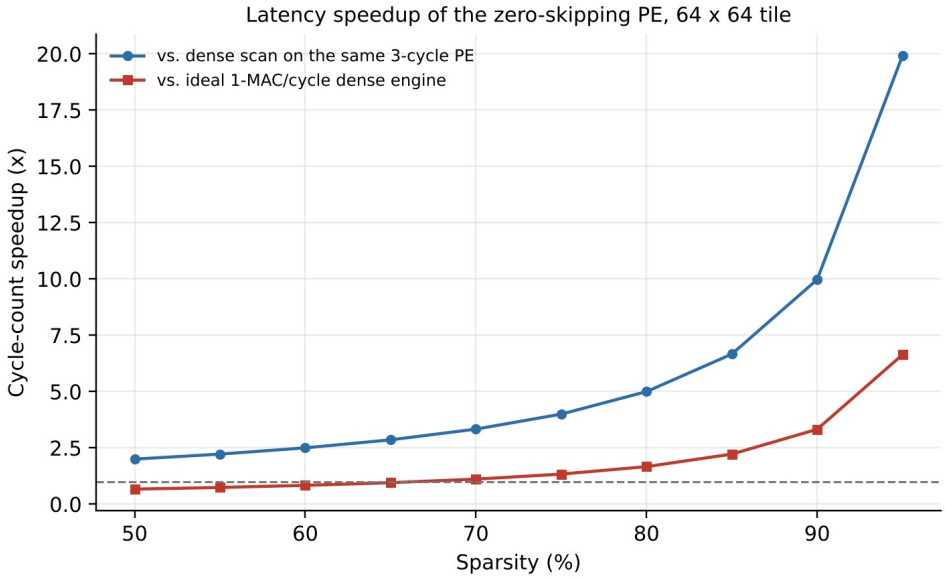


Figure 5.4 Cycle-count speedup of the zero-skipping PE versus sparsity for a 64 × 64 tile (two baselines)

(The 64 × 64 tile needs up to 2,048 non-zeros per invocation, i.e. the 11-bit instance of the parameterised design; the control flow and the equations are unchanged.)

Throughput. In steady state one PE performs one MAC (two operations: a multiply and an add) per three cycles, i.e. $2F/3$ = 64.6 MOPS of *useful* operations at 96.9 MHz. The effective throughput compares the work of the dense equivalent with the time taken, $2MN \cdot F/T \approx 2F$ / $(3(1 - s))$, which at 90 % sparsity is 2×96.9 MHz / 0.3 = 646 MOPS ≈ 0.64 GOPS: this is the effective figure reported for the proposed Sparse-CIM in Table 5.6 and in the iSES paper, and the table above shows that it grows to more than 1.2 GOPS at 95 % sparsity. Normalised by the vectorless power of 4.502 W this is about 0.14 effective GOPS/W; since the power figure is an envelope, this is a lower bound that the activity-annotated analysis of Section 5.7 is expected to improve.

Architectural comparison. Table 5.6 reproduces the comparison of the iSES paper. The entries have different scopes and precisions (full systems with FP32/INT16/INT8, an ASIC

MAC-only block, and the proposed FPGA core), so the table is read as a map of the design space rather than as a ranking.

Table 5.6 Comparison with existing sparse accelerators and MAC architectures (from the iSES paper)

Architecture	Scope / precision	Area / LUTs	Power (mW)	Delay (ns)	Throughput
GraphLily [18]	Full system / FP32	390K LUTs	43,000	6.02	35.8 GFLOPS
SERPENS [19]	Full system / FP32	173K LUTs	48,000	4.48	60.5 GFLOPS
Cerberus [20]	Full system / INT16	30.1 mm^2	17,032	1.00	112.4 GOPS
Zhang *et al.* [21]	Full system / INT8	∼120K LUTs	3,200	N/A	∼348 GOPS
Abhiram *et al.* [22]	MAC only / INT8	2967 μm^2	0.507	3.99	N/A
Proposed Sparse-CIM	**Full system / INT8**	**154 LUTs**	**4,500**	**10.32**	**0.64 GOPS (Eff.)**

Discussion. Three observations follow. (i) *Area.* The proposed core uses 154 LUTs and no DSP slices, against 120K–390K LUTs for the FPGA systems; it is a PE, not a full-system accelerator, and its role is as the replicable building block (Section 5.5). (ii) *Power.* The 4.5 W figure is a vectorless envelope for the core, which compares with 17–48 W for the data-centre-class entries; the only entry in the same INT8-FPGA class, Zhang *et al.*, reports 3.2 W for a complete 348-GOPS Vision Transformer system, which shows that the efficiency of the proposed core depends on the activity-annotated refinement described in Section 5.7 and on replication. (iii) *Delay and throughput.* The 10.32 ns path and the three-cycle schedule give 0.64 effective GOPS per PE at 90 % sparsity, below the high-bandwidth systems; the pipelining avenues of Section 5.6 and multi-PE replication (≈ 93 PEs by block-RAM count) are the path to the hundreds-of-GOPS class. The design's distinguishing property is that it reaches its operating point with no DSP slices, no off-chip memory and a single-digit-percent fraction of a low-cost device.

5.10 Measurement and characterization methodology

Every number in Chapter 5 has a defined origin. **Functional results** come from the Vivado Simulator run of the two testbenches; Table 5.1 is additionally reproduced by a cycle-accurate reference model whose agreement with the waveform at 115, 145 and 175 ns is described in Section 5.1. **Resource utilisation** is the post-synthesis utilisation report of the out-of-context core (LUTs, registers, block RAM and DSP counts). **Delay** is the post-synthesis critical-path delay of the same run. **Power** is the vectorless post-synthesis power report. **Latency, speedup and effective throughput** are analytical, computed from $T = 3 \cdot NNZ + 2$, the stated baselines and F_{max} = 1/10.32 ns. Listing 5.3 gives the Tcl flow that regenerates the synthesis-based numbers.

Listing 5.3 Vivado Tcl flow for the out-of-context characterisation

```
read_verilog -sv {bram_bank.sv mac_unit.sv sparse_cim_controller.sv sparse_cim_top.sv}
synth_design -top sparse_cim_top -part xc7z020clg484-1 -mode out_of_context
report_utilization        -file util.rpt
report_timing_summary     -file timing.rpt
report_power              -file power.rpt          ;# vectorless: no SAIF/VCD attached
```

The limits of the characterisation are stated alongside each figure: the delay and power are post-synthesis, out-of-context values; the power is a vectorless estimate with Low confidence; and throughput is an analytical figure for one PE. Section 5.11 lists the steps that turn each into a measured quantity.

5.11 Architectural scalability priorities and verification roadmap

Table 5.7 organises the system-scaling enhancements of the architecture in the order in which they build on one another, together with the verification gate that marks the completion of each.

Table 5.7 System-scaling enhancements and verification gates

Priority	Enhancement	Benefit	Verification gate
1	Multi-row CSR FSM with `row_ptr` sequencing and result write-back (Section 3.10)	Full matrix per invocation; y in on-chip bank	4 × 4 example yields (13, 3, 12, 24); $T = 3\mathrm{NNZ} + 3M + 2$
2	Self-checking scoreboard and randomised regression (Listing 3.8)	Automatic pass/fail; latency assertion	Zero mismatches over randomised CSR rows
3	AXI4-Lite wrapper and AXI BRAM controllers (Table 3.8)	Processor-controlled operation	Driver of Listing 3.10 passes in simulation and on board
4	Timing closure at the target clock (product/operand registers)	F above 100 MHz	Positive WNS in implemented design
5	Activity-annotated power (SAIF/VCD) and BRAM enable gating	Realistic power; lower activity	Power report with annotated activity
6	Zynq-7000 board deployment and measurement	Measured latency and power	On-board cycle count equals T
7	Multi-PE replication (up to ≈ 93 by BRAM count)	Near-linear throughput scaling	Aggregate GOPS versus PE count
8	Multi-precision arithmetic (INT16, FP16/BF16/FP32)	Wider applicability	Bit-exact against software reference

CHAPTER 6
CONCLUSION AND FUTURE SCOPE

6.1 Summary of the work

This project designed, verified and characterised a sparsity-aware, BRAM-centric near-memory SpMVM accelerator for edge inference on an AMD/Xilinx Zynq-7000 SoC. The computation $y = Ax$ over a pruned weight matrix is formulated in CSR, for which the storage, traffic and cycle models were derived. The architecture realises on an FPGA the near-memory counterpart of in-memory computing: a five-state zero-skipping FSM and a signed INT8 MAC with 32-bit accumulation sit directly at the synchronous read ports of three RAMB18E1-based banks that hold the compressed weights, the column indices and the dense vector. Simulation confirmed the signed accumulation $0 \rightarrow 15 \rightarrow 7$ and the control handshake, and out-of-context synthesis on the xc7z020clg484-1 gave 154 LUTs, 41 flip-flops, 3 RAMB18E1, 0 DSP48E1, a 10.32 ns critical path and a 4.502 W vectorless power estimate, with an effective throughput of 0.64 GOPS at 90 % sparsity.

6.2 Major contributions

1. A modular Sparse Compute-in-Memory / Near-Memory Processing Engine (`sparse_cim_top`) integrating three parameterised BRAM banks, a five-state zero-skipping controller and a DSP-free INT8 MAC.
2. A rigorous treatment of in-memory versus near-memory computing on commercial FPGAs, justifying BRAM-centric NMC as the practical realisation of the CIM dataflow.
3. Storage and traffic models for Dense, CSR, COO and CSC formats with index-overhead break-even thresholds, and a cycle-accurate latency model $T = 3 \cdot \mathrm{NNZ} + 2$ that follows from the one-cycle BRAM read latency.
4. A verification suite (MAC and top-level testbenches), a cycle-accurate reference trace, and a specified self-checking scoreboard extension.
5. Post-synthesis characterisation on the Zynq-7000 and a comparison with published sparse accelerators and MAC units.
6. A system-level integration roadmap: six-state multi-row CSR FSM, AXI4-Lite register map, PS–PL integration, bitstream/XSA/Vitis flow and a host driver.

6.3 Conclusions and design insights

- **Compute proportional to non-zeros.** A controller that iterates over stored non-zeros converts sparsity directly into fewer cycles, fewer memory reads and fewer MACs; at 90 % sparsity the work is one tenth of the dense work.

- **The BRAM read latency is the architecture.** The serial dependency col_idx → vec_addr → vec_data across two one-cycle synchronous reads fixes the three-cycle schedule, and the same fact points to the optimisations (overlapped fetch, pipelining) that follow.
- **LUT arithmetic is the right size for INT8.** An 8 × 8 multiplier and 32-bit accumulator fit in 154 LUTs and 41 flip-flops without DSP slices, leaving the device almost entirely free for replication; block RAM, not logic, bounds the number of PEs.
- **Near-memory is the practical optimum on FPGAs.** The hard-macro BRAM interface rules out analog in-memory computing, but placing logic at the read port delivers the CIM benefits that matter at the edge: on-chip weights, deterministic timing and no off-chip traffic.
- **Report characterisation with its scope.** The delay and power figures are post-synthesis, out-of-context values; the activity-annotated and board-level measurements of the roadmap turn them into final design numbers.

6.4 Future scope

6.4.1 Complete multi-row CSR execution and automated verification

Implement the six-state FSM of Section 3.10 with a `row_ptr` bank and a result bank, verify it against the 4 × 4 example and randomised matrices with the scoreboard of Listing 3.8, and assert the cycle count $T = 3 \cdot \mathrm{NNZ} + 3M + 2$. Add handling of empty rows and of rows longer than the bank, and constrained-random and coverage-driven verification (functional coverage of row-length, column-index and sign distributions, SystemVerilog assertions on the handshake).

6.4.2 End-to-end hardware deployment on a physical FPGA SoC (Zynq-7000 / Zynq UltraScale+)

Package the core with the AXI4-Lite wrapper (Table 3.8), integrate it in IP Integrator, export the XSA, run the driver on the ARM Cortex-A9 and measure latency with a hardware cycle counter and power with the board monitor. Port the same IP to a Zynq UltraScale+ device, where UltraRAM and a larger DSP/LUT inventory allow substantially larger tiles and more PEs, and compare the measured and estimated values.

6.4.3 Cross-accelerator and scenario-based benchmarking

Benchmark the engine on common pruned layers (for example a pruned fully connected layer and a Transformer projection) at matched sparsity, and compare it with a dense FPGA baseline, an embedded CPU running a software CSR kernel, and the published designs of Table 5.6 under a common measurement boundary (same model, same precision, same I/O accounting). Report latency, throughput, energy per inference and utilisation.

6.4.4 Multi-precision arithmetic scaling (INT16, FP16, BF16, FP32)

Parameterise the MAC for INT16 and floating-point formats (FP16, BF16, FP32) while keeping the controller and banks, so that numerically sensitive workloads such as sparse linear solves in SLAM can use wider arithmetic and quantisation-sensitive networks can use mixed precision. The study should quantify the LUT/DSP trade-off for each format and the extra accumulator width needed. In the opposite direction, sub-byte INT4 and 2:4 structured sparsity [15] reduce index and storage overhead further.

6.4.5 Extended journal publication roadmap (Targeting IEEE TVLSI, IEEE TCAS-I/II, or ACM TRETS)

An extended journal article can combine the architecture and storage/latency models of this report with the completed multi-row execution, the implemented and measured results on a Zynq board, the pipelined schedule and the cross-accelerator benchmark. Suitable venues are IEEE Transactions on VLSI Systems, IEEE Transactions on Circuits and Systems I/II, and ACM Transactions on Reconfigurable Technology and Systems.

VISIBLE OUTCOME

The project has produced the technical deliverables summarised in Table 6.1.

Table 6.1 Visible technical deliverables

Deliverable	Recorded outcome
RTL modules	`bram_bank.sv`, `mac_unit.sv`, `sparse_cim_controller.sv`, `sparse_cim_top.sv` (SystemVerilog)
Testbenches	`tb_mac_unit.sv`, `tb_sparse_cim_top.sv`; Vivado Simulator waveform showing 0 → 15 → 7
Synthesis characterisation	Zynq-7000 xc7z020clg484-1: 154 LUTs, 41 FDCE, 3 RAMB18E1, 0 DSP48E1; 10.32 ns critical path; 4.502 W vectorless power; 0.64 GOPS effective
Analytical models	CSR/COO/CSC storage, break-even densities, $T = 3 \cdot NNZ + 2$ latency, speedup and throughput tables
Conference paper	IEEE iSES conference paper “A Sparsity-Aware Near-Memory SpMVM Accelerator for Edge Inference on FPGA” (camera-ready manuscript)
Integration roadmap	Six-state multi-row FSM, AXI4-Lite register map, Zynq PS–PL block design, XSA/Vitis workflow, host driver

Publication status. The architecture, the RTL, and the post-synthesis characterisation have been reported in an IEEE conference paper authored by Vemana Venkata Pujithram and Dr. Rahul Ratnakumar (Manipal Institute of Technology, Manipal). The extended journal article outlined in Section 6.4.5 will build on the completed multi-row execution and board measurements.

REFERENCES

[1] N. P. Jouppi *et al.*, "In-Datacenter Performance Analysis of a Tensor Processing Unit," in *Proc. ACM/IEEE 44th Annu. Int. Symp. Computer Architecture (ISCA)*, 2017, pp. 1–12.

[2] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both Weights and Connections for Efficient Neural Network," *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015.

[3] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, "Sparsity in Deep Learning: Pruning and Growth for Efficient Inference and Training in Neural Networks," *J. Machine Learning Research*, vol. 22, no. 241, pp. 1–124, 2021.

[4] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.

[5] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, 2014, pp. 10–14.

[6] Xilinx/AMD, *Zynq-7000 SoC Technical Reference Manual*, UG585.

[7] Xilinx/AMD, *Zynq-7000 SoC Data Sheet: Overview*, DS190.

[8] Xilinx/AMD, *7 Series FPGAs Memory Resources User Guide*, UG473.

[9] B. Jacob *et al.*, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[10] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A Survey of Quantization Methods for Efficient Neural Network Inference," arXiv:2103.13630, 2021.

[11] Xilinx/AMD, *7 Series DSP48E1 Slice User Guide*, UG479.

[12] N. Verma *et al.*, "In-Memory Computing: Advances and Prospects," *IEEE Solid-State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, 2019.

[13] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, "Memory devices and applications for in-memory computing," *Nature Nanotechnology*, vol. 15, pp. 529–544, 2020.

[14] C.-J. Jhang, C.-X. Xue, J.-M. Hung, F.-C. Chang, and M.-F. Chang, "Challenges and Trends of SRAM-Based Computing-In-Memory for AI Edge Devices," *IEEE Trans. Circuits Syst. I*, vol. 68, no. 5, pp. 1773–1786, May 2021.

[15] A. Mishra *et al.*, "Accelerating Sparse Deep Neural Networks," arXiv:2104.08378, 2021.

[16] A. Parashar *et al.*, "SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks," in *Proc. ACM/IEEE 44th Annu. Int. Symp. Computer Architecture (ISCA)*, 2017, pp. 27–40.

[17] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "EIE: Efficient Inference Engine on Compressed Deep Neural Network," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Computer Architecture (ISCA)*, 2016, pp. 243–254.

[18] Y. Hu, Y. Du, E. Ustun, and Z. Zhang, "GraphLily: Accelerating Graph Linear Algebra on HBM-Equipped FPGAs," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2021.

[19] L. Song, Y. Chi, L. Guo, and J. Cong, "SERPENS: A High Bandwidth Memory Based Accelerator for General-Purpose Sparse Matrix-Vector Multiplication," in *Proc. 59th ACM/IEEE Design Automation Conf. (DAC)*, 2022, pp. 211–216.

[20] S. Hwang, D. Baek, J. Park, and J. Huh, “Cerberus: Triple Mode Acceleration of Sparse Matrix and Vector Multiplication,” *ACM Trans. Archit. Code Optim.*, vol. 21, no. 2, Art. 38, May 2024.

[21] Y. Zhang, L. Feng, H. Shan, and Z. Zhu, “A 109-GOPs/W FPGA-Based Vision Transformer Accelerator With Weight-Loop Dataflow Featuring Data Reusing and Resource Saving,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 12, pp. 13596–13610, Dec. 2024.

[22] K. M. Abhiram, S. B. Bore Gowda, and R. Ratnakumar, “High Performance Fault-Resilient MAC Unit for Mission-Critical AI Hardware,” School of Electrical Engineering, MIT Manipal, 2024.

[23] Y. Liu *et al.*, “FPGA-Based Sparse Matrix Multiplication Accelerators: From State-of-the-Art to Future Opportunities,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 17, no. 4, Art. 59, Nov. 2024.

[24] LeapSpace, “Hardware accelerator for Sparse Matrix-Vector Multiplication,” ScienceDirect Research Summary, 2026.

[25] M. S. Kumar, D. A. Kumar, and P. Samundiswary, “Design and performance analysis of Multiply-Accumulate (MAC) unit,” in *Proc. Int. Conf. Circuits, Power and Computing Technologies*, 2014.

[26] M. Pavan Kumar and D. V. S. Vijaya Lakshmi, “Hybrid MAC Implementation Using Booth and Wallace Approximate Adders and Multipliers,” *Int. J. of Science and Research*, 2023.

[27] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel, “Optimization of Sparse Matrix-Vector Multiplication on Emerging Multicore Platforms,” in *Proc. ACM/IEEE Conf. Supercomputing (SC)*, 2007.

[28] N. Bell and M. Garland, “Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors,” in *Proc. Conf. High Performance Computing Networking, Storage and Analysis (SC)*, 2009.

[29] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks,” in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Computer Architecture (ISCA)*, 2016, pp. 367–379.

[30] Y. Umuroglu *et al.*, “FINN: A Framework for Fast, Scalable Binarized Neural Network Inference,” in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2017, pp. 65–74.

[31] Xilinx/AMD, *7 Series FPGAs Configurable Logic Block User Guide*, UG474.

[32] Xilinx/AMD, *Vivado Design Suite User Guide: Synthesis*, UG901.

[33] Xilinx/AMD, *Vivado Design Suite User Guide: Using Constraints*, UG903.

[34] Xilinx/AMD, *Vivado Design Suite User Guide: Power Analysis and Optimization*, UG907.

[35] ARM Ltd., *AMBA AXI and ACE Protocol Specification (AXI3, AXI4, AXI4-Lite)*, ARM IHI 0022.

[36] *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.

[37] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA, USA: SIAM, 2003.

[38] Xilinx/AMD, *Vivado Design Suite User Guide: Creating and Packaging Custom IP*, UG1118.

[39] Xilinx/AMD, *AXI BRAM Controller LogiCORE IP Product Guide*, PG078.

[40] Xilinx/AMD, *Vivado Design Suite User Guide: Designing IP Subsystems Using IP Integrator*, UG994.

[41] AMD, *Vitis Unified Software Platform Documentation: Embedded Software Development*, UG1400.

MINOR SPECIALIZATION PROJECT DETAILS

Field	Details
Student name	Vemana Venkata Pujithram
Registration number	230907016
Branch / Section / Roll number	Electronics and Communication Engineering / _____ / _____
Mail ID / Mobile	vemana.mitmpl2023@learner.manipal.edu / _____

Project and guide	Details
Project title	A Sparsity-Aware Near-Memory SpMVM Accelerator for Edge Inference on FPGA
Project type	Individual Minor Specialization Project
Minor specialization	Minor in Embedded Systems
Guide name and designation	Dr. Rahul Ratnakumar, Assistant Professor
Guide department	Department of Electronics and Communication Engineering, MIT Manipal
Target device and toolchain	AMD/Xilinx Zynq-7000 SoC (xc7z020clg484-1); Xilinx Vivado

Sign-off	Signature	Date
Student: Vemana Venkata Pujithram (230907016)		
Guide: Dr. Rahul Ratnakumar		
Head of Department		